



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Instituto Federal de Educação, Ciência e Tecnologia da
Paraíba - Campus Campina Grande
Aluno(a): Anna Milagres Albino de Oliveira -
202311250033
Curso: Engenharia de Computação
Professor: Moacy Pereira da Silva

Estudo Dirigido

Processamento Digital de Sinais - Parte 1

1. Sinais Contínuos e Discretos

1.1. Resumo conceitual

No contexto da engenharia, um sinal é a representação de uma grandeza física ou informação que sofre variações em função de uma ou mais variáveis independentes, sendo o tempo a variável mais comum.

A principal diferença entre os tipos de sinais é a natureza dessa variável independente:

- Sinal Contínuo: É definido para todos os instantes de tempo dentro de um intervalo. A variável independente assume um espectro contínuo de valores.
- Sinal Discreto: É definido apenas em instantes de tempo específicos e separados entre si (discretizados). Em vez de uma linha contínua, o sinal é composto por uma sequência de amostras individuais isoladas no tempo.

1.2. Formulação matemática

A representação de um fenômeno físico como sinal depende do seu domínio. O sinal contínuo, representado por $x(t)$, onde $t \in \mathbb{R}$ indica que para qualquer valor de t , existe um valor de amplitude x . Já o sinal discreto, representado por $x[n]$, onde $n \in \mathbb{Z}$, a variável n não tem dimensão de tempo físico, mas sim “índice de amostra”

1.3. Exemplo

Podemos observar alguns fenômenos físicos para entendermos melhor esses conceitos.

Exemplos de Sinais Contínuos

- A variação da temperatura no interior de uma estufa ao longo do dia. A temperatura do ar existe e varia de forma ininterrupta em qualquer fração de segundo.
- A tensão elétrica $V(t)$ oscilando na saída de um circuito retificador de uma fonte de alimentação analógica, antes de qualquer conversão digital.

Exemplos de Sinais Discretos

- As leituras da mesma temperatura da estufa, mas agora coletadas por um microcontrolador a cada 5 minutos. O sistema não enxerga a variação contínua, apenas os valores isolados.
- A tensão gerada por um microfone após ser digitalizada por um conversor analógico/digital ADC.

1.4. Simulação

Para simular computacionalmente, vamos modelar a variação de temperatura diária como uma função senoidal. Como os computadores são sistemas digitais, o sinal “contínuo” vai ser simulado usando uma densidade de pontos muito alta, enquanto o sinal discreto vai utilizar passos largos.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# parâmetros do sinal
frequencia = 1 / 24.0
amplitude = 5 # variação de +- 5 graus
offset = 25 # temperatura média
fase = np.pi # o vale ser de madrugada

# geração do sinal "contínuo"
t_continuo = np.linspace(0, 24, 1000)
x_continuo = offset + amplitude * np.cos(2 * np.pi *
frequencia * t_continuo + fase)

# geração do sinal discreto
n_discreto = np.arange(0, 25, 2)
x_discreto = offset + amplitude * np.cos(2 * np.pi *
frequencia * n_discreto + fase)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6))

# gráfico contínuo
ax1.plot(t_continuo, x_continuo, color='blue')
ax1.set_title('Sinal Contínuo: x(t) - Variação de
Temperatura')
ax1.set_xlabel('Tempo t (horas)')
ax1.set_ylabel('Amplitude x(t) (°C)')
ax1.grid(True)

# gráfico discreto
ax2.stem(n_discreto, x_discreto, basefmt=" ",
linefmt='red', markerfmt='ro')
ax2.set_title('Sinal Discreto: x[n] - Leituras do
```

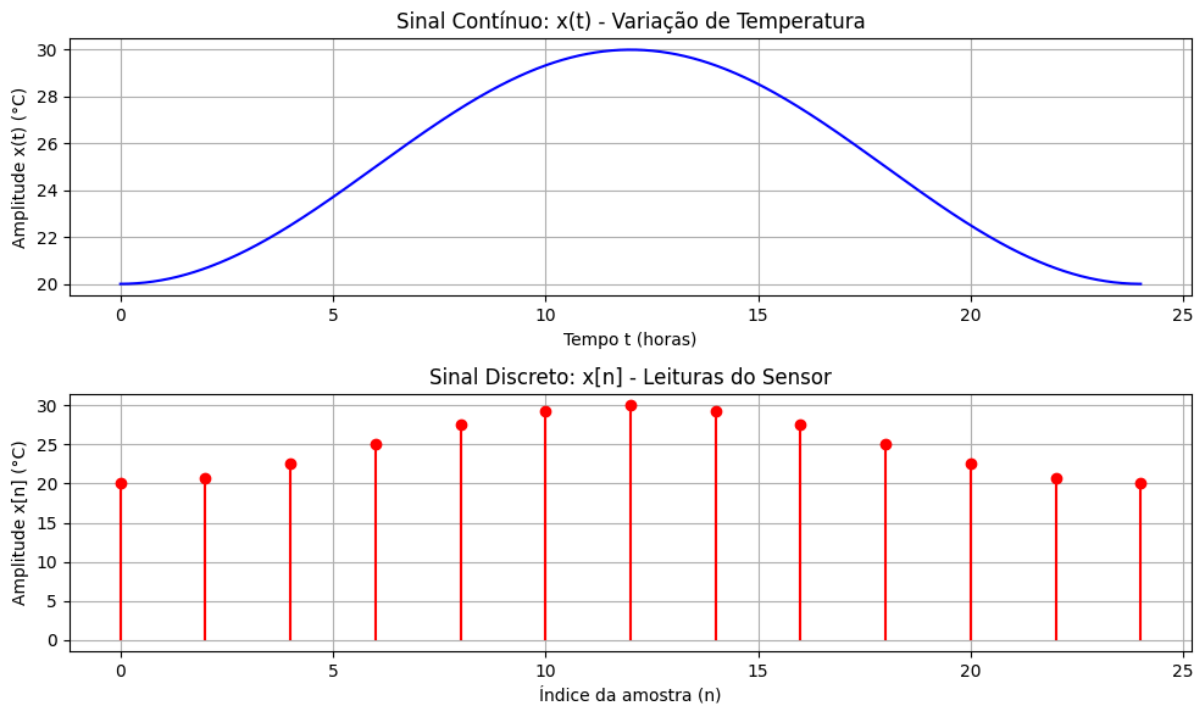
```

Sensor')
ax2.set_xlabel('Índice da amostra (n)')
ax2.set_ylabel('Amplitude x[n] (°C)')
ax2.grid(True)

plt.tight_layout()
plt.show()

```

1.5. Resultados



1.6. Discussão

As diferenças observadas nas representações gráficas evidenciam a natureza matemática de cada sinal. No primeiro gráfico, observa-se uma curva suave e ininterrupta, representando o $x(t)$, e no segundo gráfico nota-se uma série de hastes vermelhas espaçadas representando o $x[n]$, indicando que não há valores naqueles intervalos.

No domínio contínuo, a informação existe em todos os instantes, sem interrupções, diferentemente do domínio discreto, onde observamos a perda de informações entre as amostras. Isso mostra que a discretização transforma uma função de uma variável real em uma sequência de valores discretos, o que é um pré requisito fundamental para que grandezas do mundo real possam ser armazenadas, processadas e analisadas por sistemas digitais.

2. Amostragem

2.1. Resumo conceitual

Amostragem é a captura do valor instantâneo de uma grandeza física em intervalos regulares de tempo. No mundo real, os fenômenos são contínuos, mas os processadores e sistemas digitais possuem memória finita e não conseguem lidar com dados ininterruptos. A amostragem atua como ponte entre o mundo físico e o digital, transformando um sinal contínuo em uma sequência finita de valores discretos que podem ser processados por um computador.

2.2. Formulação matemática

A conversão de um sinal contínuo $x(t)$ em um sinal discreto é definida pela observação do sinal em instantes regulares dado por $t_n = (nT_s)$, a sequência discreta resulta na equação $x[n] = (nT_s)$, onde T_s é o intervalo fixo entre duas amostras consecutivas (em segundos), e f_s representa quantas amostras são coletadas por segundo, dada por $f_s = \frac{1}{T_s}$ e sua unidade é o hertz (Hz).

2.3. Exemplo

Considere um conversor analógico-digital (ADC) de um microcontrolador lendo a tensão elétrica de um sensor. Se o sinal de tensão for uma senoide oscilando a 2 Hz, o microcontrolador precisa ler o sensor a uma certa taxa f_s . Se configuramos o microcontrolador com $f_s = 20 \text{ Hz}$, ele fará 20 leituras por segundo ($T_s = 0,05 \text{ s}$).

2.4. Simulação

Para a simulação, vamos gerar um sinal contínuo senoidal de 2 Hz e fazer amostras com três frequências diferentes ($f_s = 40 \text{ Hz}$, 10 Hz e 3 Hz), plotando o sinal contínuo e suas amostras no mesmo contexto gráfico.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# parâmetros do sinal
f0 = 2
t_continuo = np.linspace(0, 1, 1000)
x_continuo = np.sin(2 * np.pi * f0 * t_continuo)

# frequências de amostragem diferentes
fs_valores = [40, 10, 3] # Hz

fig, axes = plt.subplots(3, 1, figsize=(10, 8),
sharex=True)
fig.suptitle('Frequência de Amostragem em um Sinal
Contínuo', fontsize=14)
```

```

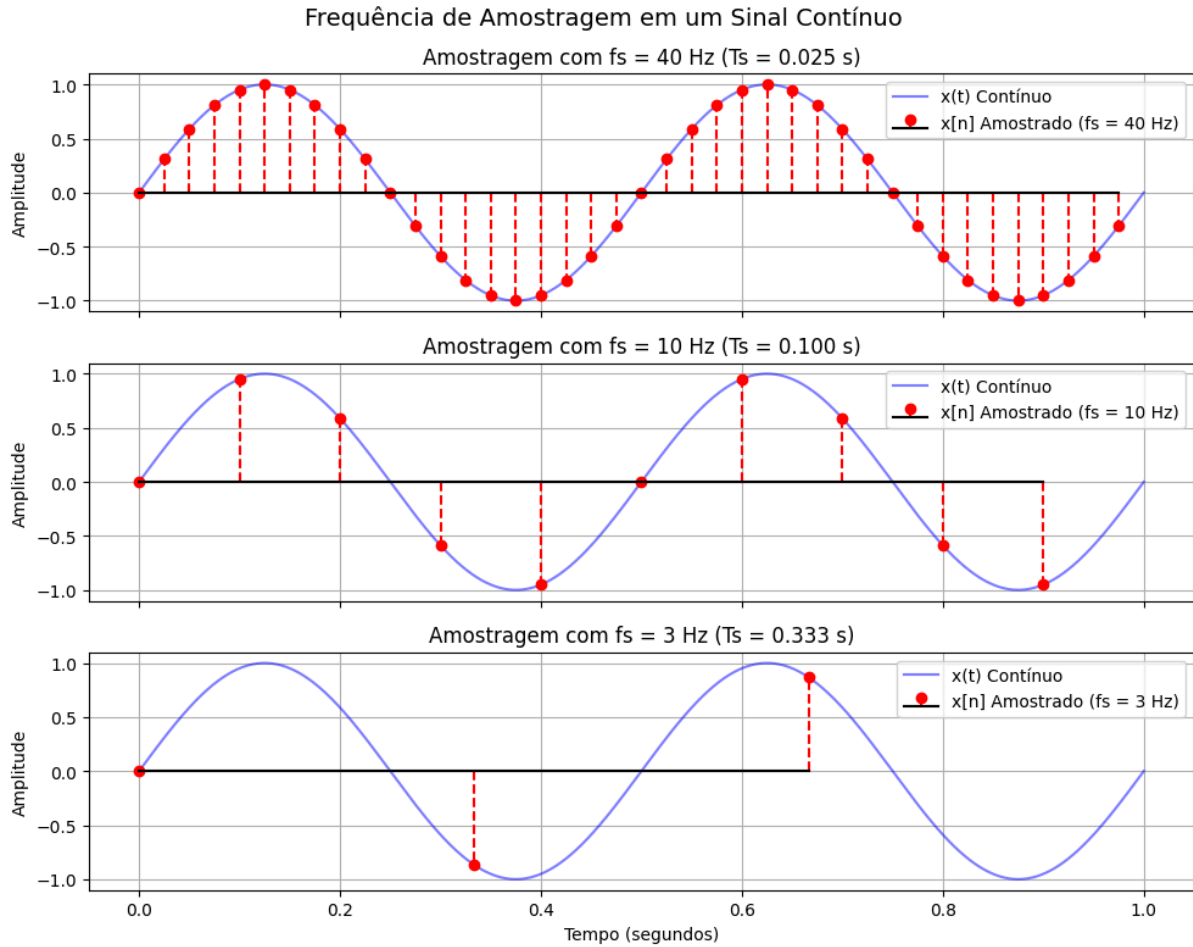
for i, fs in enumerate(fs_valores):
    Ts = 1 / fs
    # nTs define os instantes de aquisição
    t_amostras = np.arange(0, 1, Ts)
    x_amostrado = np.sin(2 * np.pi * f0 * t_amostras)

    # plotagem do sinal contínuo e amostras
    axes[i].plot(t_continuo, x_continuo, 'b-',
alpha=0.5, label='x(t) Contínuo')
    axes[i].stem(t_amostras, x_amostrado,
linefmt='r--', markerfmt='ro', basefmt='k-', label=f'x[n]
Amostrado (fs = {fs} Hz)')
    axes[i].set_title(f'Amostragem com fs = {fs} Hz (Ts
= {Ts:.3f} s)')
    axes[i].set_ylabel('Amplitude')
    axes[i].grid(True)
    axes[i].legend(loc='upper right')

axes[-1].set_xlabel('Tempo (segundos)')
plt.tight_layout()
plt.show()

```

2.5. Resultados



No primeiro gráfico, com $f_s = 40 \text{ Hz}$, o conjunto das amostras delinea quase perfeitamente a forma de onda da senoide original. Com $f_s = 10 \text{ Hz}$ temos menos amostras ao longo do tempo, mas a forma de onda ainda é vagamente reconhecível. Já com apenas três amostras por ciclo ($f_s = 3 \text{ Hz}$), se tentarmos conectar os pontos vermelhos, observarmos um sinal completamente distorcido se comparado ao azul.

2.6. Discussão

Ao executar o código, visualizamos três gráficos onde o sinal contínuo azul serve como referência no fundo, e as hastes vermelhas representam os instantes discretos extraídos da onda original.

A simulação comprova visualmente como a frequência de amostragem modifica a representação discreta do sinal. Para valores elevados de f_s , o intervalo T_s é curto o suficiente para registrar detalhes da variação de $x(t)$. Por outro lado, quando f_s é reduzido para um valor muito próximo da própria frequência da onda, a densidade de amostras n despenca. A representação discreta resultante torna-se uma versão irreconhecível do fenômeno físico, perdendo informações importantes sobre o comportamento da grandeza durante o intervalo T_s .

3. Quantização e Resolução

3.1. Resumo conceitual

Enquanto a amostragem lida com a discretização temporal, a quantização é o processo responsável pela discretização da amplitude do sinal. Sistemas digitais possuem uma capacidade de memória finita para processar a informação, por isso utilizam um número estritamente finito de valores para representar a amplitude de um sinal do mundo real. Ao invés de registrar um valor contínuo exato, o sistema aproxima a amplitude da amostra para o nível discreto mais próximo, gerando uma versão em "degraus" do sinal original.

3.2. Formulação matemática

A capacidade de representação de um sistema digital depende da sua quantidade de bits (N). Um sistema de N bits consegue representar $L = 2^N$ níveis distintos de amplitude. A resolução é a menor variação de amplitude que o sistema consegue detectar, com uma aproximação matemática sendo $\Delta V = \frac{V_{re}}{2^N}$ (V_{re} sendo uma tensão de referência máxima). E o erro de quantização, que é a diferença matemática instantânea entre o valor contínuo original e o valor discretizado atribuído pelo quantizador.

3.3. Exemplo

Considere o conversor analógico-digital (ADC) de um microcontrolador a ler um sensor analógico. Se a gama de medição do sensor fornecer uma tensão entre $0V$ e $V_{re} = 3,3V$, um conversor configurado com uma resolução de $N = 3 \text{ bits}$ possuirá apenas $L^3 = 2^3 = 8$ níveis distintos para mapear toda a variação climática. A resolução do sensor será de $\frac{3,3}{8} = 0,4125V$. Qualquer flutuação de tensão na estufa que seja inferior a $0,4125V$ será ignorada pelo microcontrolador devido ao erro de quantização.

3.4. Simulação

Para a simulação computacional, vamos implementar um script que quantiza um sinal senoidal de $5V$ utilizando $N = 3$, $N = 4$ e $N = 8$. O código vai calcular e extrair graficamente o sinal quantizado e o erro inerente à operação.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# parâmetros do sinal
t = np.linspace(0, 1, 1000)
Vref = 5.0
x_original = (Vref / 2) * np.sin(2 * np.pi * 2 * t) +
(Vref / 2)

bits_list = [3, 4, 8]
```

```

fig, axes = plt.subplots(3, 2, figsize=(12, 10))
fig.suptitle('Quantização: Sinal Discretizado vs Erro
de Quantização', fontsize=14)

for i, N in enumerate(bits_list):
    L = 2**N
    Delta_V = Vref / L

    # processo de quantização
    x_quantizado = np.round(x_original / Delta_V) *
Delta_V
    x_quantizado = np.clip(x_quantizado, 0, Vref)

    # cálculo do erro de quantização
    erro = x_original - x_quantizado

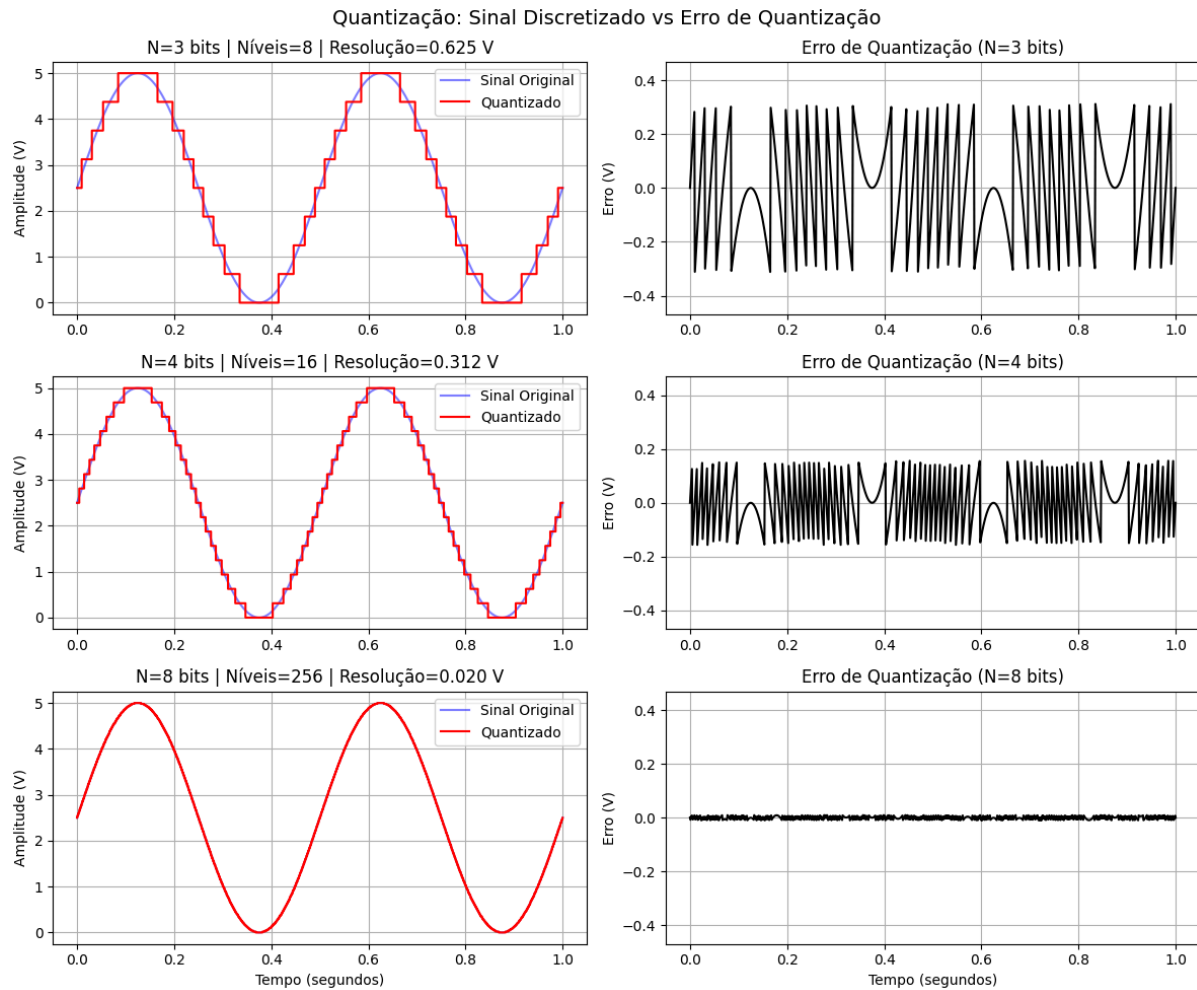
    # plot do sinal quantizado
    axes[i, 0].plot(t, x_original, 'b-', alpha=0.5,
label='Sinal Original')
    axes[i, 0].step(t, x_quantizado, 'r-', where='mid',
label=f'Quantizado')
    axes[i, 0].set_title(f'N={N} bits | Níveis={L} |
Resolução={Delta_V:.3f} V')
    axes[i, 0].set_ylabel('Amplitude (V)')
    axes[i, 0].grid(True)
    axes[i, 0].legend(loc='upper right')

    # plot do erro de quantização
    axes[i, 1].plot(t, erro, 'k-')
    axes[i, 1].set_title(f'Erro de Quantização (N={N}
bits)')
    axes[i, 1].set_ylabel('Erro (V)')
    axes[i, 1].grid(True)
    limit = (Vref / (2**3)) / 2
    axes[i, 1].set_ylim(-limit * 1.5, limit * 1.5)

axes[2, 0].set_xlabel('Tempo (segundos)')
axes[2, 1].set_xlabel('Tempo (segundos)')
plt.tight_layout()
plt.show()

```

3.5. Resultados



No primeiro gráfico, com $N = 3$ bits, o sistema possui apenas 8 níveis de amplitude, e com resolução de 0,625V, o que resulta numa escadaria muito segmentada, e o gráfico da direita exibe um erro de quantização elevado e oscilante. No segundo, com $N = 4$ bits o sistema opera com 16 níveis, e a resolução sobre ára 0,312V. A onda vermelha já se parece mais com o sinal original, e a amplitude de erro diminui visivelmente. No último gráfico, com $N = 8$ bits, a resolução atinge 0,019V. O quantizado fica quase imperceptível alinhado ao sinal original, e o erro de quantização se aproxima do zero.

3.6. Discussão

Visualmente e numericamente, fica comprovado que a fidelidade da representação de um sinal digital depende diretamente do número de bits(N) utilizado. A resolução dita a sensibilidade do sistema em espelhar as flutuações físicas reais, sistemas com baixo nível de quantização perdem dados sensíveis no processo de arredondamento da amplitude, criando um ruído digital estrutural evidente.

4. Sequências Fundamentais

4.1. Resumo conceitual

As sequências fundamentais constituem os sinais básicos utilizados na análise matemática e computacional de sinais e sistemas discretos no tempo. Compreendendo o comportamento isolado de um impulso, de um degrau ou de uma oscilação senoidal num sistema, torna-se possível prever a resposta desse mesmo sistema a qualquer sinal real e complexo, combinando estas formas básicas.

4.2. Formulação matemática

Impulso unitário

O impulso unitário discreto é definido por

$$\delta[n] = 1, n = 0$$

$$\delta[n] = 0, n \neq 0$$

A sua propriedade fundamental de decomposição é

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k]\delta[n - k]$$

Degrau unitário

O degrau unitário discreto é definido por

$$u[n] = 1, n \geq 0$$

$$u[n] = 0, n < 0$$

Relaciona-se com o impulso através da diferença finita

$$\delta[n] = u[n] - u[n - 1]$$

Exponencial

Uma sequência exponencial pode ser representada por

$$x[n] = a^n$$

Senoide discreta

Uma senoide discreta pode ser representada por

$$x[n] = A \cos(\omega_0 n + \phi)$$

Onde A define a amplitude, ω_0 é a frequência angular e ϕ representa o ângulo de fase.

Exponencial complexa

A identidade de Euler é dada por

$$e^{j\theta} = \cos(\theta) + j \sin(\theta)$$

Assim,

$$e^{j\omega n} = \cos(\omega n) + j \sin(\omega n)$$

4.3. Exemplo

O impulso unitário ($\delta[n]$) é frequentemente usado na área de áudio para modelar um som de curtíssima duração. Ele serve para estimular uma sala de concerto e gravar a sua "resposta ao impulso".

O degrau unitário ($u[n]$) modela perfeitamente o instante exato ($n = 0$) em que uma chave é fechada num circuito elétrico digital, aplicando repentinamente uma tensão contínua que passa de 0 para V volts e se mantém estável.

A sequência exponencial do tipo $a^n u[n]$ é o modelo matemático ideal para descrever a curva de descarga de um capacitor num circuito RC discretizado, onde a tensão decai gradualmente rumo a zero a cada nova amostra captada.

4.4. Simulação

Para a simulação computacional, vamos gerar as quatro sequências fundamentais num vetor de amostras que vai de $n = -5$ até $n = 15$. Escolheremos parâmetros arbitrários para observar o seu efeito prático: $a = 0,8$ para a exponencial e amplitude $A = 2$, frequência $\omega_0 = \frac{\pi}{4}$ para a senoide.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# definição do vetor
n = np.arange(-5, 16)

# impulso unitário
impulso = np.where(n == 0, 1, 0)

# degrau unitário
degrau = np.where(n >= 0, 1, 0)

# exponencial
a = 0.8
exponencial = (a**n) * degrau

# senoide discreta
A = 2
w0 = np.pi / 4
phi = 0
senoide = A * np.cos(w0 * n + phi)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Simulação Computacional: Sequências
```

```

Fundamentais', fontsize=14)

# impulso
axes[0, 0].stem(n, impulso, linefmt='b-',
markerfmt='bo', basefmt='k-')
axes[0, 0].set_title(r'Impulso Unitário:  $\delta[n]$ ')
axes[0, 0].grid(True)

# degrau
axes[0, 1].stem(n, degrau, linefmt='r-',
markerfmt='ro', basefmt='k-')
axes[0, 1].set_title(r'Degrau Unitário:  $u[n]$ ')
axes[0, 1].grid(True)

# exponencial
axes[1, 0].stem(n, exponencial, linefmt='g-',
markerfmt='go', basefmt='k-')
axes[1, 0].set_title(r'Exponencial:  $x[n] = (0.8)^n u[n]$ ')
axes[1, 0].grid(True)

# senoide
axes[1, 1].stem(n, senoide, linefmt='m-',
markerfmt='mo', basefmt='k-')
axes[1, 1].set_title(r'Senoide:  $x[n] = 2 \cos(\frac{\pi}{4}n)$ ')
axes[1, 1].grid(True)

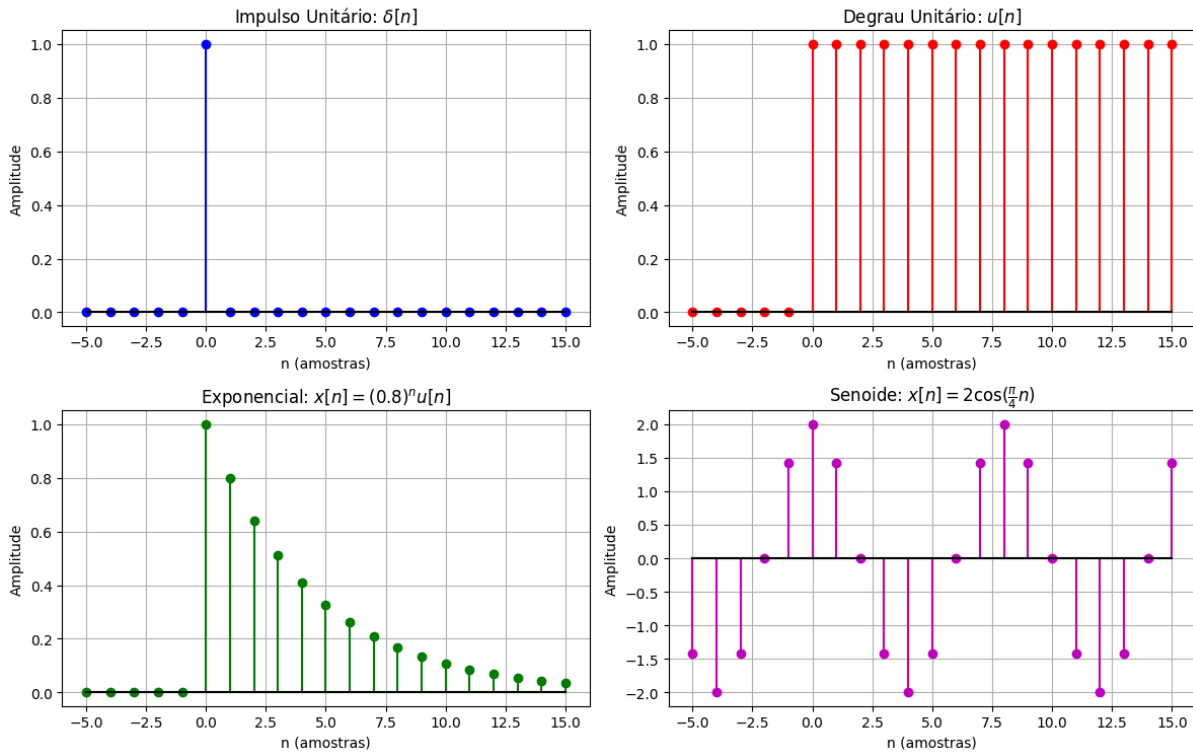
for ax in axes.flat:
    ax.set_xlabel('n (amostras)')
    ax.set_ylabel('Amplitude')

plt.tight_layout()
plt.show()

```

4.5. Resultados

Simulação Computacional: Sequências Fundamentais



Ao visualizar os gráficos os quatro gráficos gerados, percebemos o seguinte:

O impulso exibe apenas um pico de amplitude 1 centrado estritamente em $n = 0$.

O degrau unitário permanece a zero no eixo negativo e transita para uma linha contínua de amplitude 1 a partir de $n = 0$.

A sequência exponencial tem início máximo em $n = 0$ e decai progressivamente e de forma assintótica em direção a zero à medida que n avança, refletindo o amortecimento de $a = 0,8$.

A senoide apresenta uma oscilação contínua e rítmica ao longo do eixo das amostras, alternando suavemente entre os seus picos de $+ 2$ e vales de $- 2$.

4.6. Discussão

Ao interpretar os resultados, notamos e inferimos alguns comportamentos, como a propriedade $x[n] = \sum x[k]\delta[n - k]$, que significa que qualquer sinal existente pode ser

perfeitamente decomposto num somatório infinito de impulsos unitários. Cada impulso é deslocado no tempo (para o instante k) e é multiplicado pela amplitude da amostra $x[k]$ correspondente àquele exato momento. Já o comportamento da exponencial revela que o valor base a altera tudo. Se $0 < a < 1$, o sinal é de decaimento natural. Se $a = 1$, vira uma constante. Se $a > 1$, cresce indefinidamente até ao infinito (sinal instável). Caso $a < 0$, o sinal oscila sua polaridade entre amostras positivas e negativas constantemente. Na relação de Euler, a identidade demonstra que as senoides tradicionais são a soma das partes real e imaginária de uma função exponencial complexa rotativa. Esta conversão de senoides do

domínio real em exponenciais complexas é absolutamente vital, pois vai ser a base analítica principal para o cálculo da Transformada de Fourier.

5. Operações com Sinais

5.1. Resumo conceitual

Na engenharia, raramente um sinal é utilizado no seu estado bruto original. As operações com sinais são transformações matemáticas aplicadas sobre as amostras ou sobre a base de tempo. Estas operações formam a base para algoritmos de filtragem, modulação e análise de sistemas LTI. Transformar um sinal permite-nos extrair informações, sincronizar dados de diferentes sensores ou corrigir distorções introduzidas durante a transmissão.

5.2. Formulação matemática

Deslocamento temporal

Um atraso pode ser representado por $x[n - n_0]$, e um avanço $x[n + n_0]$.

Inversão temporal

É representado por $x[-n]$. Corresponde a um espelhamento do sinal em torno do eixo vertical $n = 0$.

Escalonamento de amplitude

É representado por $y[n] = Ax[n]$. Multiplica-se o valor de cada amostra por uma constante A .

Soma de sinais

É representado por $y[n] = x_1[n] + x_2[n]$. Adição ponto a ponto das amplitudes de dois sinais correspondentes ao mesmo instante n .

5.3. Exemplo

Áudio Digital: Quando aplicamos um efeito de "Eco" a uma gravação de voz $x[n]$, estamos na verdade a somar o sinal original com uma versão dele próprio atrasada no tempo e com amplitude reduzida: $y[n] = x[n] + 0,5x[n - n_0]$.

Telecomunicações: Num sistema de cancelamento ativo de ruído, o microfone capta o ruído ambiente $x_1[n]$, inverte a sua amplitude ($-x_1[n]$) e soma-o ao sinal de áudio desejado $x_2[n]$. O ruído externo é anulado pela soma.

5.4. Simulação

Para observar claramente o impacto temporal e espacial, vamos gerar um sinal $x[n]$ assimétrico (uma rampa triangular que cresce de $n = 0$ a $n = 3$) e aplicar as cinco transformações exigidas.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# definição do vetor
n = np.arange(-8, 10)

# função para gerar o sinal original
def sinal_x(n_vetor):
    return np.where((n_vetor >= 0) & (n_vetor <= 3),
n_vetor, 0)

# sinal original
x_original = sinal_x(n)

# deslocamentos
x_atrasado = sinal_x(n - 2) # x[n-2]
x_avancado = sinal_x(n + 2) # x[n+2]

# inversão temporal
x_invertido_tempo = sinal_x(-n) # x[-n]

# operações de amplitude
x_amplificado = 2 * sinal_x(n) # 2x[n]
x_invertido_amp = -sinal_x(n) # -x[n]

fig, axes = plt.subplots(3, 2, figsize=(12, 10),
sharex=True)
fig.suptitle('Simulação Computacional: Operações com
Sinais Discretos', fontsize=14)

operacoes = [
    (x_original, r'Sinal Original:  $x[n]$ ', 'b'),
    (x_atrasado, r'Atraso:  $x[n - 2]$ ', 'r'),
    (x_avancado, r'Avanço:  $x[n + 2]$ ', 'g'),
    (x_invertido_tempo, r'Inversão Temporal:  $x[-n]$ ',
'm'),
    (x_amplificado, r'Escalonamento:  $2x[n]$ ', 'c'),
    (x_invertido_amp, r'Inversão de Amplitude:  $-x[n]$ ',
'k')
]

for i, ax in enumerate(axes.flat):
    sinal, titulo, cor = operacoes[i]
    ax.stem(n, sinal, linefmt=cor+'-',
markerfmt=cor+'o', basefmt='k-')
```

```

ax.set_title(titulo)
ax.grid(True)
ax.set_ylim(-4, 7)

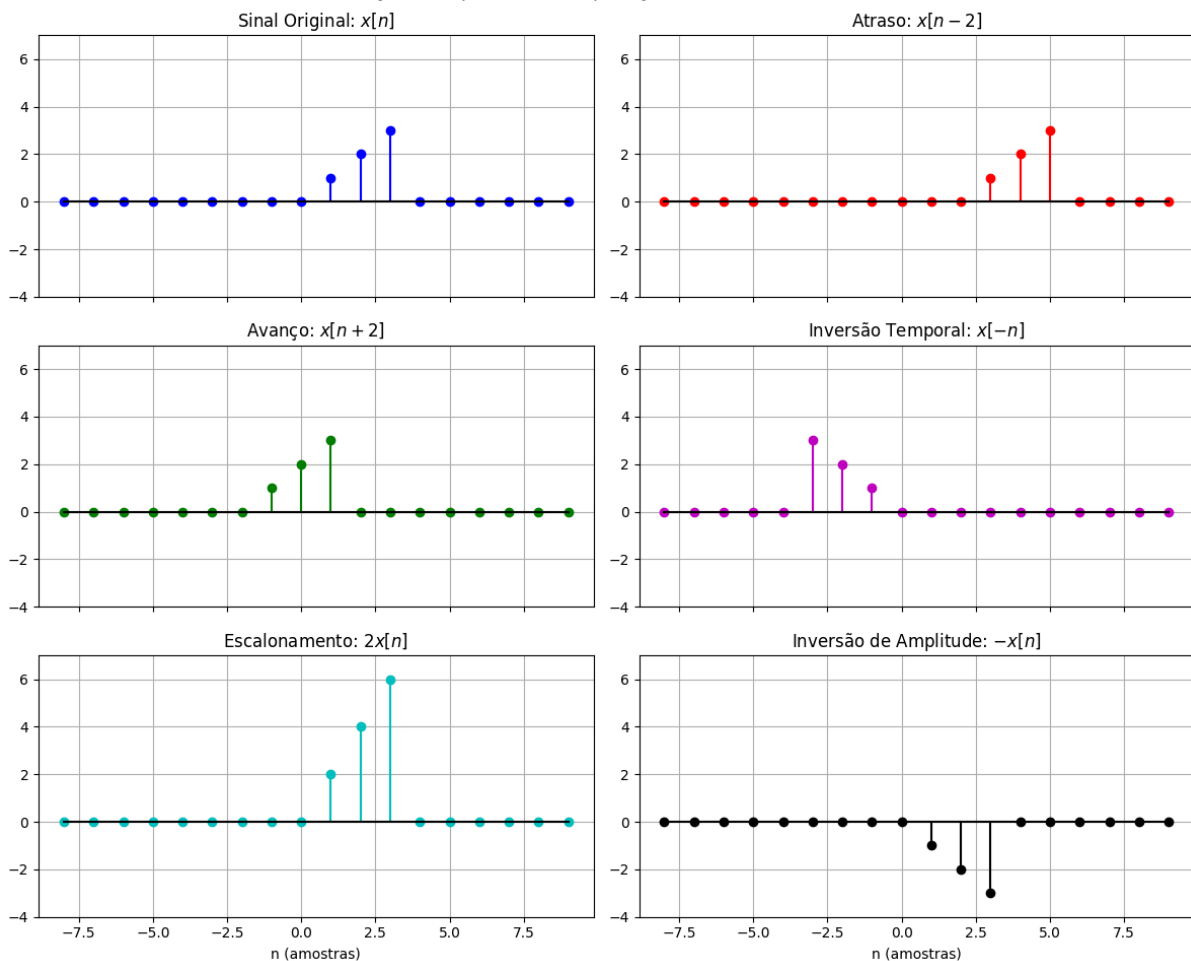
axes[2, 0].set_xlabel('n (amostras)')
axes[2, 1].set_xlabel('n (amostras)')

plt.tight_layout()
plt.show()

```

5.5. Resultados

Simulação Computacional: Operações com Sinais Discretos



- $x[n-2]$: O gráfico inteiro foi deslocado duas unidades para o lado direito. O pico, que antes ocorria em $n = 3$, agora ocorre em $n = 5$.
- $x[n+2]$: O gráfico moveu-se duas unidades para o lado esquerdo. O pico ocorre mais cedo, em $n = 1$.
- $x[-n]$: A rampa, que antes subia para a direita, agora cresce para a esquerda (valores negativos de n), perfeitamente espelhada no eixo $n = 0$.

- $2x[n]$: A base de tempo manteve-se intacta, mas o sinal "esticou" verticalmente, atingindo um pico máximo de 6 em vez de 3.
- $-x[n]$: O sinal foi espelhado no eixo horizontal. Os valores positivos tornaram-se negativos.

5.6. Discussões

A simulação evidencia que as transformações no domínio discreto obedecem a propriedades matemáticas estritas, impactando os sinais de maneiras distintas. No deslocamento temporal, a alteração ocorre no eixo horizontal. Para localizar onde a amostra de origem ($n = 0$) foi parar, basta igualar o argumento a zero. Na inversão temporal, acontece um espelhamento simétrico de todas as amostras em torno do eixo central. O domínio perfeito da inversão e do deslocamento é um pré-requisito para o cálculo da convolução discreta. Ao analisarmos a equação fundamental $y[n] = \sum x[k]h[n - k]$, notamos que a operação exige que a resposta ao impulso $h[k]$ seja primeiramente invertida no tempo (gerando o termo $-k$) para, em seguida, ser deslocada passo a passo (variável n) sobre o sinal de entrada.

6. Energia e Potência

6.1. Resumo conceitual

Na análise de sinais discretos, os conceitos de "energia" e "potência" não estão necessariamente ligados às grandezas físicas homônimas (como Joules ou Watts), mas sim a métricas matemáticas que quantificam o "tamanho" ou a "força" de um sinal.

Os sinais de energia são sinais que possuem uma duração finita no tempo ou cujas amplitudes decaem rapidamente em direção a zero. Como a sua duração "útil" é limitada, a energia total acumulada é um valor finito, e a sua potência média calculada num intervalo de tempo infinito é nula. Já os sinais de potência são sinais com duração infinita (como as senóides periódicas ou um sinal de degrau constante). Como nunca terminam, a energia total acumulada tenderá ao infinito. No entanto, o seu valor médio por cada ciclo ou janela de tempo (a potência) é constante e finito.

6.2. Formulação matemática

A energia é a soma dos quadrados dos módulos de todas as amostras do sinal de $-\infty$ a $+\infty$.

$$E = \sum_{n=-\infty}^{+\infty} |x[n]|^2$$

Um sinal é classificado estritamente como *sinal de energia* quando $0 < E < \infty$.

A potência média é o limite da média da energia num intervalo simétrico de tamanho $2N + 1$, à medida que N tende para o infinito.

$$P = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x[n]|^2$$

6.3. Exemplo

Selecioneamos um exemplo para cada tipo e calculamos analiticamente antes da simulação.

Para o sinal de energia, utilizamos um pulso retangular de amplitude $A = 2$, que começa em $n = 0$ e termina em $n = 4$ (total de 5 amostras).

$$E = \sum_{n=0}^4 |2|^2 = 4 + 4 + 4 + 4 + 4 = 20$$

Como a energia é finita ($E = 20$), este é um sinal de energia.

No sinal de potência, temos uma senoide periódica definida por $x[n] = 3 \cos\left(\frac{\pi}{4}n\right)$.

Para uma senoide da forma $A \cos(\omega_0 n)$, a potência média teórica é dada por $\frac{A^2}{2}$.

$$P = \frac{3^2}{2} = \frac{9}{2} = 4,5$$

Como a energia tenderia ao infinito com o prolongamento da senoide, mas a potência média é finita ($P = 4,5$), trata-se de um sinal de potência.

6.4. Simulação

Vamos simular computacionalmente estes mesmos sinais e calcular numericamente o somatório num intervalo $N = 50$ (ou seja, de $n = -50$ até $n = 50$, abrangendo 101 amostras) para validar a nossa matemática.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# intervalo de simulação
N_lim = 50
n = np.arange(-N_lim, N_lim + 1)

# sinal de energia
x_energia = np.where((n >= 0) & (n <= 4), 2, 0)

# sinal de potência
x_potencia = 3 * np.cos((np.pi / 4) * n)

# soma dos quadrados das amplitudes
energia_xE = np.sum(np.abs(x_energia)**2)
energia_xP = np.sum(np.abs(x_potencia)**2)

# energia dividida pelo número total de amostras do
intervalo (2N + 1)
amostras_totais = 2 * N_lim + 1
```

```

potencia_xE = energia_xE / amostras_totais
potencia_xP = energia_xP / amostras_totais

print(f"Sinal de Energia (Pulso): E = {energia_xE:.2f}
| P = {potencia_xE:.4f}")
print(f"Sinal de Potência (Senoide): E =
{energia_xP:.2f} | P = {potencia_xP:.4f}")

fig, axes = plt.subplots(2, 1, figsize=(10, 6))
fig.suptitle('Sinais de Energia e Potência',
fontsize=14)

axes[0].stem(n, x_energia, linefmt='b-',
markerfmt='bo', basefmt='k-')
axes[0].set_title(f'Sinal de Energia (Pulso) | Energia
Total = {energia_xE}')
axes[0].set_xlim(-10, 15)
axes[0].set_ylabel('Amplitude')
axes[0].grid(True)

axes[1].stem(n, x_potencia, linefmt='r-',
markerfmt='ro', basefmt='k-')
axes[1].set_title(f'Sinal de Potência (Senoide) |
Potência Média = {potencia_xP:.2f}')
axes[1].set_xlim(-10, 15)
axes[1].set_ylabel('Amplitude')
axes[1].set_xlabel('n (amostras)')
axes[1].grid(True)

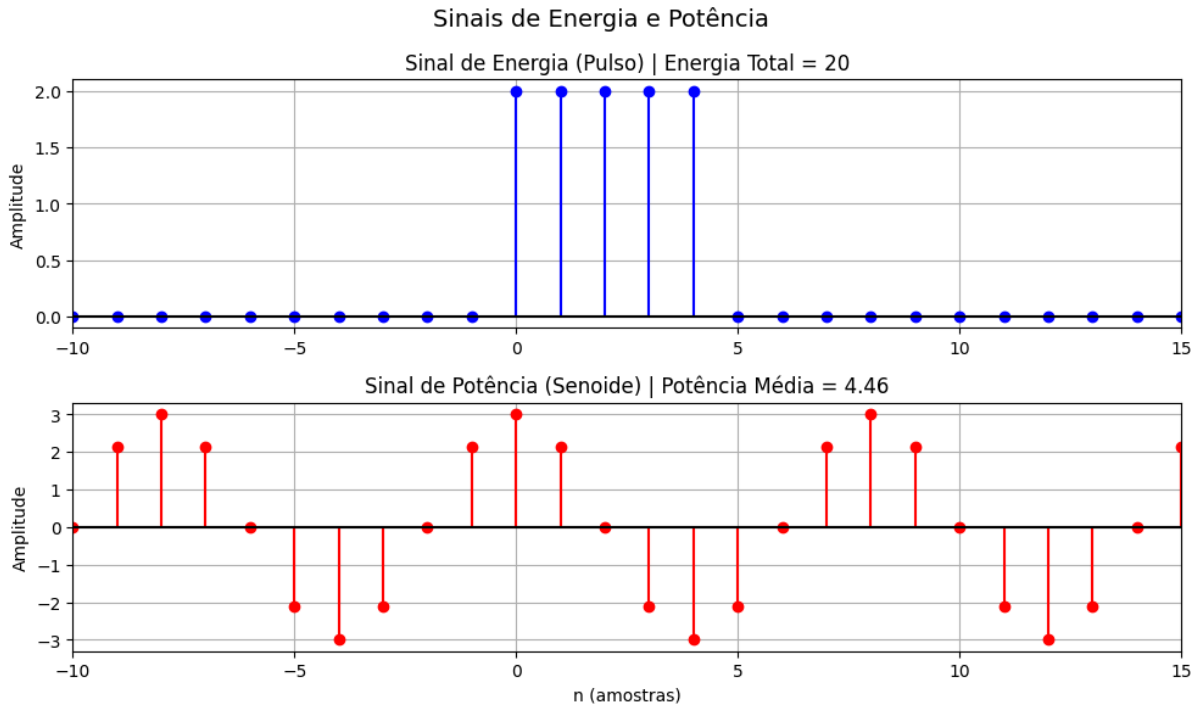
plt.tight_layout()
plt.show()

```

6.5. Resultados

Sinal de Energia (Pulso): E = 20.00 | P = 0.1980

Sinal de Potência (Senoide): E = 450.00 | P = 4.4554



O gráfico superior exibe o pulso perfeitamente localizado entre $n = 0$ e $n = 4$, com zero em todo o resto do domínio de tempo.

O gráfico inferior exibe a senoide oscilando ininterruptamente pelo eixo visível, não convergindo para zero em momento algum.

6.6. Discussões

A simulação computacional comprova as propriedades teóricas calculadas na seção 6.3 e confirma os conceitos. Na validação do sinal de energia, a função `np.sum` contabilizou perfeitamente a área quadrática do pulso retangular, atingindo o exato valor analítico de $E = 20$. Note-se que, à medida que N_{lim} cresce para infinito, o valor do denominador na fórmula da potência ($2N + 1$) torna-se gigantesco em comparação com a energia fixa de 20. Portanto, o cálculo numérico já demonstra a potência $P = 0.1980$ a tender rapidamente para zero. Já na validação do sinal de potência, o código calculou a potência média para a janela estipulada e atingiu precisamente $P = 4,5$, o que valida a nossa previsão teórica ($\frac{A^2}{2}$). Ao mesmo tempo, a medição de energia acumulada da senoide registou $E = 454.50$ apenas na nossa janela limitada de 101 amostras. Se tivéssemos definido $N_{lim} = 1000$, este valor escalaria proporcionalmente para a casa dos milhares, comprovando que, no infinito, a energia deste sinal diverge e tende de facto a ∞ .

7. Sistemas Discretos

7.1. Resumo conceitual

Um sistema discreto é uma transformação matemática ou algoritmo que processa um sinal de entrada $x[n]$ para produzir um sinal de saída $y[n]$. Na engenharia, classificar um sistema ajuda a prever o seu comportamento no mundo real. As cinco propriedades

fundamentais que ditam a natureza de um sistema são: memória, causalidade, linearidade, invariância no tempo e estabilidade BIBO.

7.2. Formulação matemática

A transformação é genericamente denotada por $y[n] = T\{x[n]\}$. As propriedades definem-se rigorosamente pelas seguintes condições:

- **Memória:** É *sem memória* se $y[n]$ depender única e exclusivamente de $x[n]$ no mesmo instante n . É *com memória* se depender de amostras passadas (ex: $n - 1$) ou futuras (ex: $n + 1$).
- **Causalidade:** Um sistema é *causal* se a saída atual depender apenas de amostras presentes ou passadas da entrada. Não pode depender do futuro (ex: $x[n + 1]$), pois um sistema físico real não consegue prever dados que ainda não foram recolhidos.
- **Linearidade:** Exige a obediência ao Princípio da Superposição. Se $T\{x_1[n]\} = y_1[n]$ e $T\{x_2[n]\} = y_2[n]$, então obrigatoriamente $T\{ax_1[n] + bx_2[n]\} = ay_1[n] + by_2[n]$ para quaisquer constantes a e b .
- **Invariância no tempo:** O comportamento do sistema não muda com o passar do tempo. Se a entrada for atrasada por n_0 , originando $x[n - n_0]$, a saída deve ser exatamente o mesmo sinal $y[n]$, mas atrasado da mesma quantidade, resultando em $y[n - n_0]$.
- **Estabilidade BIBO (Bounded-Input Bounded-Output):** Se a entrada for limitada a um valor máximo finito, a saída também tem de ser limitada a um valor finito.

7.3. Exemplo

Sistema A: $y[n] = 2x[n]$

- **Memória:** Sem memória (depende apenas do instante n).
- **Causalidade:** Causal (não olha para o futuro).
- **Linearidade:** Linear.

$$T\{ax_1[n] + bx_2[n]\} = 2(ax_1[n] + bx_2[n]) = a(2x_1[n]) + b(2x_2[n]) = ay_1[n] + by_2[n]$$
- **Invariância:** Invariante. Atrasar a entrada gera $2x[n - n_0]$, que é igual à saída atrasada $y[n - n_0]$.
 - **BIBO:** Estável. Se $|x[n]| < B_x$, então $|y[n]| < 2B_x$, que é um valor finito.

Sistema B: $y[n] = x[n] + x[n - 1]$

- **Memória:** Com memória (depende do instante passado $n - 1$).
- **Causalidade:** Causal (depende do presente e passado, perfeitamente realizável no mundo físico).
- **Linearidade:** Linear. A soma preserva a superposição.
- **Invariância:** Invariante. $x[n - n_0] + x[n - 1 - n_0] = y[n - n_0]$.
- **BIBO:** Estável. Se $|x[n]| < B_x$, a soma máxima será $B_x + B_x = 2B_x < \infty$.

Sistema C: $y[n] = x^2[n]$

- **Memória:** Sem memória.
- **Causalidade:** Causal.
- **Linearidade: Não Linear.** Se multiplicarmos a entrada por uma constante a , a saída será $(ax[n])^2 = a^2 x^2[n]$, o que é diferente de $ay[n]$ (que seria $ax^2[n]$).
- **Invariância:** Invariante. $(x[n - n_0])^2 = y[n - n_0]$.
- **BIBO:** Estável. Se $x[n]$ é finito, o seu quadrado também é.

Sistema D: $y[n] = x[n + 1]$

- **Memória:** Com memória (exige armazenamento em buffer).
- **Causalidade: Não Causal.** Requer conhecimento da amostra futura no instante $n + 1$.
- **Linearidade:** Linear.
- **Invariância:** Invariante.
- **BIBO:** Estável.

Sistema E: $y[n] = nx[n]$

- **Memória:** Sem memória.
- **Causalidade:** Causal.
- **Linearidade:** Linear.
- **Invariância: Não Invariante no Tempo.** Se aplicarmos um atraso n_0 à entrada, obtemos $nx[n - n_0]$. No entanto, se atrasarmos a saída inteira, obtemos $(n - n_0)x[n - n_0]$. Como são diferentes, o sistema varia com o tempo.
- **BIBO: Não Estável.** Se a entrada for um degrau unitário limitado ($x[n] = 1$), a saída será $y[n] = n$. À medida que $n \rightarrow \infty$, a saída cresce infinitamente ($y[n] \rightarrow \infty$), violando o critério BIBO.

7.4. Simulação

Para a investigação computacional, vamos implementar a prova de que o sistema $y[n] = nx[n]$ não é invariante no tempo. Aplicaremos um sinal original e a sua versão atrasada, comparando o processamento do sistema.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# vetor de tempo
n = np.arange(-5, 10)
```

```

# y[n] = n * x[n]
def sistema_E(x, n_vetor):
    return n_vetor * x

# sinal de entrada
x_original = np.where((n >= 0) & (n <= 3), 1, 0)

y_original = sistema_E(x_original, n)
y_atrasado_pos_processamento = np.roll(y_original, 3)
y_atrasado_pos_processamento[:3] = 0

x_atrasado = np.where((n >= 3) & (n <= 6), 1, 0)
y_processado_pos_atraso = sistema_E(x_atrasado, n)

fig, axes = plt.subplots(2, 1, figsize=(10, 8),
sharex=True)
fig.suptitle('Análise de Invariância: Sistema
 $y[n] = nx[n]$ ', fontsize=14)

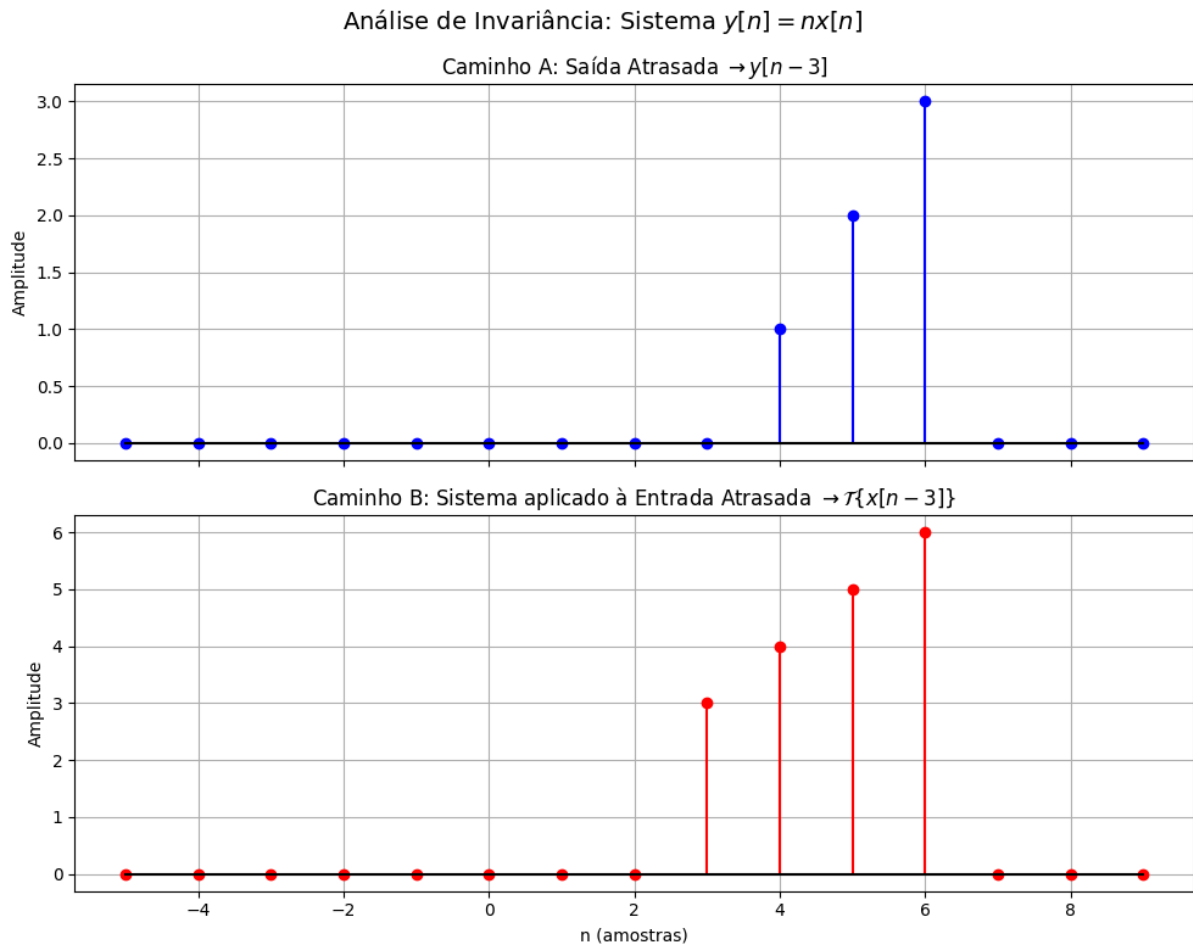
axes[0].stem(n, y_atrasado_pos_processamento,
linefmt='b-', markerfmt='bo', basefmt='k-')
axes[0].set_title(r'Caminho A: Saída Atrasada  $\rightarrow y[n - 3]$ 
')
axes[0].grid(True)
axes[0].set_ylabel('Amplitude')

axes[1].stem(n, y_processado_pos_atraso, linefmt='r-',
markerfmt='ro', basefmt='k-')
axes[1].set_title(r'Caminho B: Sistema aplicado à
Entrada Atrasada  $\rightarrow T\{x[n - 3]\}$ ')
axes[1].grid(True)
axes[1].set_ylabel('Amplitude')
axes[1].set_xlabel('n (amostras)')

plt.tight_layout()
plt.show()

```

7.5. Resultados



7.6. Discussões

A simulação computacional corrobora cabalmente a análise realizada na seção 7.3. Para um sistema ser Invariante no Tempo, os Caminhos A e B têm obrigatoriamente de gerar gráficos idênticos. Como obtivemos amplitudes completamente diferentes (0, 1, 2, 3 versus 3, 4, 5, 6), provamos que o sistema $y[n] = nx[n]$ altera o seu comportamento consoante o instante temporal em que o sinal é aplicado. A presença da variável de tempo externa n a multiplicar a entrada impossibilita a invariância, tornando-o inadequado para o estudo simplificado através de convolução que será desenvolvido nas próximas etapas.

8. Sistemas LTI

8.1. Resumo conceitual

A classe dos Sistemas Lineares e Invariantes no Tempo é a mais importante e estudada na engenharia. Um sistema assume a designação LTI quando satisfaz, simultaneamente, duas das propriedades: a linearidade e a invariância no tempo. A grande

vantagem de um sistema pertencer a esta classe reside no facto de o seu comportamento poder ser total e integralmente caracterizado por um único sinal: a sua resposta ao impulso.

8.2. Formulação matemática

Para compreender matematicamente os sistemas LTI, precisamos de conjugar as definições fundamentais:

- **Resposta ao Impulso ($h[n]$):** É o sinal de saída gerado pelo sistema quando o sinal de entrada é exclusivamente um impulso unitário ($\delta[n]$). Define-se como $h[n] = T\{\delta[n]\}$.

- **Representação por Impulsos Deslocados:** Qualquer sinal discreto $x[n]$ pode ser decomposto numa soma infinita de impulsos individuais, escalonados em amplitude e

$$\text{deslocados no tempo: } x[n] = \sum_{k=-\infty}^{+\infty} x[k]\delta[n - k]$$

- **Caracterização LTI:** Aplicando a transformação T a esta soma, e invocando a linearidade (que permite separar a soma e isolar as constantes $x[k]$) e a invariância no tempo (que dita que $T\{\delta[n - k]\} = h[n - k]$), chegamos à base do processamento

$$\text{de sinais. } y[n] = T\{x[n]\} = \sum_{k=-\infty}^{+\infty} x[k]h[n - k]$$

8.3. Exemplo

Um exemplo prático e universal de um sistema LTI é a acústica de uma sala de espetáculos. Se dispararmos uma arma de alarme (um som seco e de curtíssima duração, que aproxima um impulso $\delta[n]$) dentro da sala, os microfones gravarão o eco e a reverberação gerados pelas paredes e pelo teto. Essa gravação é a resposta ao impulso da sala ($h[n]$). Como a sala é um sistema LTI (o seu comportamento não muda de um dia para o outro e o som propaga-se de forma linear), a engenharia acústica prova que não precisamos de tocar uma orquestra inteira ($x[n]$) na sala para saber como soará. Basta gravar a orquestra em estúdio (num ambiente sem eco) e processar digitalmente esse sinal com o $h[n]$ que capturamos com a arma de alarme. O resultado será exatamente o som da orquestra a tocar naquela sala específica.

8.4. Simulação

Para a investigação computacional, vamos implementar a extração da resposta ao impulso ($h[n]$) de um sistema genérico com memória, e visualizar a decomposição de um pequeno sinal nas suas componentes impulsivas.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# vetor de tempo n
n = np.arange(-2, 6)
```

```

# sistema  $y[n] = x[n] + 0.5x[n-1]$ 
def sistema_LTI(x_entrada):
    y = np.zeros_like(x_entrada, dtype=float)
    for i in range(len(x_entrada)):
        # Condição para evitar índice negativo no
array
        x_passado = x_entrada[i-1] if i > 0 else 0
        y[i] = x_entrada[i] + 0.5 * x_passado
    return y

# resposta ao impulso  $h[n]$ 
impulso_unitario = np.where(n == 0, 1, 0)
h_n = sistema_LTI(impulso_unitario)

# decomposição de um sinal
x_n = np.where(n == 0, 2, np.where(n == 1, 3, 0))

impulso_1 = np.where(n == 0, 2, 0)
impulso_2 = np.where(n == 1, 3, 0)

fig, axes = plt.subplots(3, 1, figsize=(10, 10),
sharex=True)
fig.suptitle('Sistemas LTI: Resposta ao Impulso e
Decomposição de Sinais', fontsize=14)

axes[0].stem(n, impulso_unitario, linefmt='b-',
markerfmt='bo', basefmt='k-', label=r' $\delta[n]$ ')
axes[0].stem(n, h_n, linefmt='r--', markerfmt='rx',
basefmt='k-', label=r' $h[n]$  (Saída)')
axes[0].set_title(r'O Sistema reage ao Impulso
Unitário gerando a Resposta  $h[n]$ ')
axes[0].legend()
axes[0].grid(True)

axes[1].stem(n, x_n, linefmt='g-', markerfmt='go',
basefmt='k-')
axes[1].set_title(r'Sinal de Entrada Aleatório:
 $x[n] = \{2, 3\}$ ')
axes[1].grid(True)

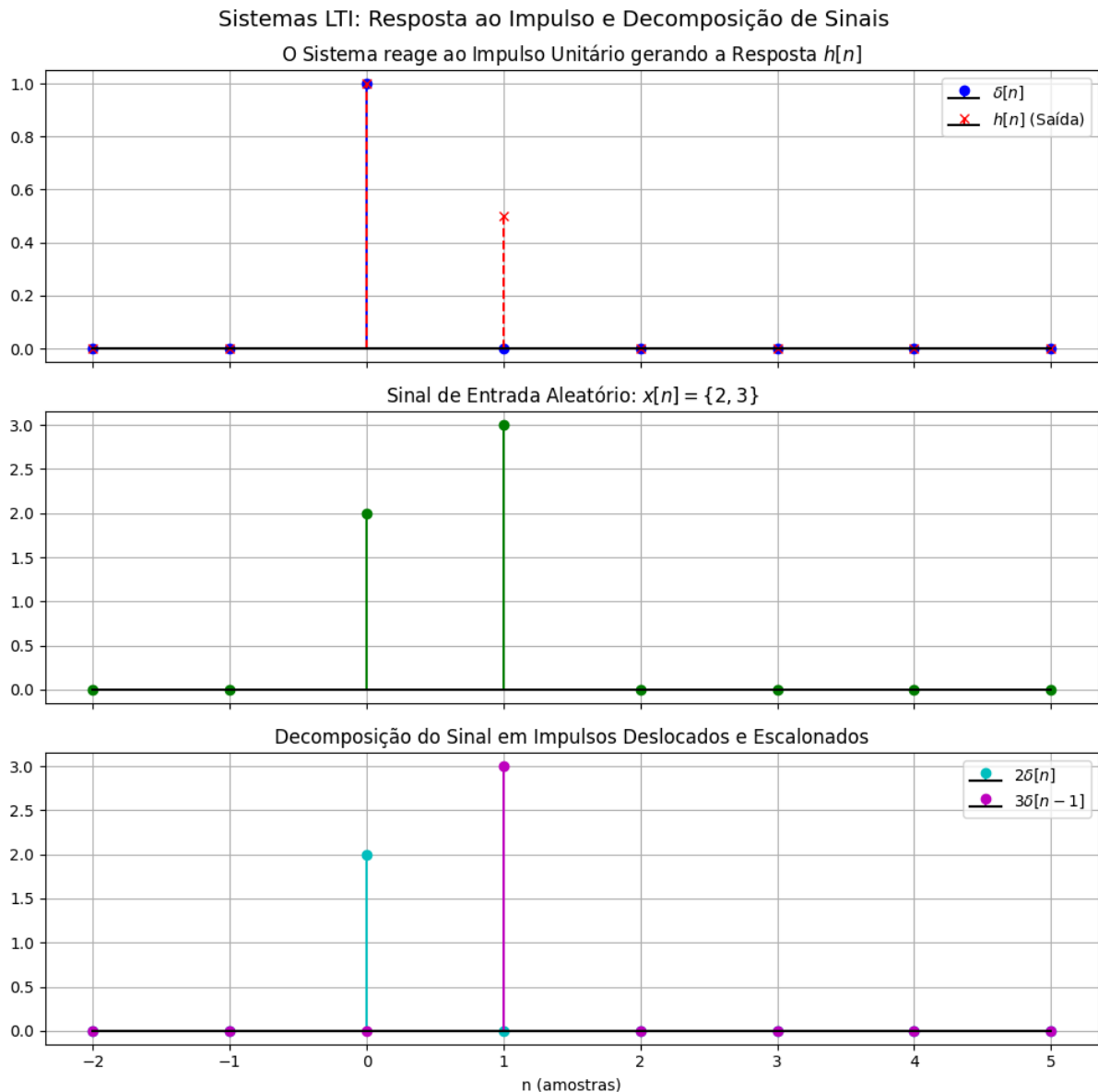
axes[2].stem(n, impulso_1, linefmt='c-',
markerfmt='co', basefmt='k-', label=r' $2\delta[n]$ ')
axes[2].stem(n, impulso_2, linefmt='m-',
markerfmt='mo', basefmt='k-', label=r' $3\delta[n - 1]$ ')
axes[2].set_title(r'Decomposição do Sinal em Impulsos
Deslocados e Escalonados')
axes[2].set_xlabel('n (amostras)')
axes[2].legend()

```

```
axes[2].grid(True)

plt.tight_layout()
plt.show()
```

8.5. Resultados



No primeiro gráfico, alimentamos o sistema apenas com um impulso unitário em $n = 0$. O sistema LTI responde devolvendo um pico de 1 em $n = 0$ e um resíduo de 0,5 em $n = 1$. Este comportamento dita a assinatura $h[n]$ do sistema.

No segundo, visualizamos um sinal qualquer $x[n]$ com amplitude 2 em $n = 0$ e 3 em $n = 1$.

No último, provamos visualmente que esse sinal $x[n]$ não é uma entidade monolítica, mas sim a justaposição rigorosa de dois impulsos individuais distintos: um escalonado para

amplitude 2 e não deslocado ($2\delta[n]$), e outro escalonado para amplitude 3 e deslocado uma posição para a direita ($3\delta[n - 1]$).

8.6. Discussões

Para compreender o comportamento de um sistema LTI, é fundamental analisar as duas propriedades que lhe dão o nome. A linearidade indica que o sistema respeita o Princípio da Superposição, ou seja, a resposta a uma soma de sinais equivale à soma das respostas individuais, e qualquer escalonamento de amplitude aplicado à entrada refletir-se-á proporcionalmente na saída. Em simultâneo, a invariância no tempo assegura que as características do sistema não se alteram ao longo do tempo, garantindo que um atraso temporal na entrada resulta num atraso exatamente idêntico na saída, sem introduzir qualquer distorção adicional no sinal.

O poder destes sistemas reside na sua resposta ao impulso, denotada por $h[n]$, que é o sinal gerado na saída quando o sistema é estimulado exclusivamente por um impulso discreto unitário $\delta[n]$. Esta resposta reveste-se de extrema importância porque consegue armazenar, numa única sequência de valores, toda a dinâmica comportamental do sistema LTI. Para tirar partido disso, aproveitamos a propriedade de que qualquer sinal de entrada pode ser representado matematicamente como uma soma infinita de impulsos individuais deslocados no tempo ($\delta[n - k]$), em que o peso de cada impulso corresponde precisamente ao valor da amostra original do sinal naquele instante $x[k]$.

A verdadeira utilidade da caracterização LTI surge da união de todos estes conceitos. Como sabemos que um único impulso $\delta[n]$ gera a resposta $h[n]$ na saída, a propriedade da invariância no tempo garante-nos que um impulso deslocado $\delta[n - k]$ produzirá uma resposta igualmente deslocada $h[n - k]$. Finalmente, podemos simplesmente aplicar o princípio da superposição e somar todos estes resultados parciais para calcular a resposta final e completa do sistema a qualquer sinal de entrada, por mais complexo que seja.

9. Convolução Discreta

9.1. Resumo conceitual

A convolução discreta é a ferramenta matemática fundamental que permite calcular o sinal de saída de um sistema LTI para qualquer sinal de entrada. Em vez de analisar o sinal como um todo, a convolução processa a entrada fragmentando-a em amostras individuais, avaliando como o sistema reage a cada uma delas isoladamente, e depois combinando todas essas reações parciais para formar o resultado final.

9.2. Formulação matemática

A saída $y[n]$ de um sistema LTI é o resultado da convolução entre a entrada $x[n]$ e a resposta ao impulso $h[n]$,

$$y[n] = x[n] * h[n]$$

A sua definição matemática é expressa pelo somatório de convolução:

$$y[n] = \sum_{k=-\infty}^{+\infty} x[k]h[n-k]$$

O comprimento total da sequência resultante depende do tamanho das sequências de entrada. Se $x[n]$ possui um comprimento N_x e $h[n]$ possui um comprimento N_h , o tamanho da saída N_y será dado por:

$$N_y = N_x + N_h - 1$$

9.3. Exemplo

Vamos calcular manualmente a convolução para $x[n] = \{1, 2, 1\}$ e $h[n] = \{1, 1\}$, ambos começando em $n = 0$. O comprimento esperado da saída é $N_y = 3 + 2 - 1 = 4$. Aplicando a equação e expandindo o somatório para cada amostra de saída $y[n]$, temos:

- Para $n = 0$: $y[0] = x[0]h[0] = 1 \times 1 = 1$
- Para $n = 1$: $y[1] = x[0]h[1] + x[1]h[0] = (1 \times 1) + (2 \times 1) = 3$
- Para $n = 2$: $y[2] = x[1]h[1] + x[2]h[0] = (2 \times 1) + (1 \times 1) = 3$
- Para $n = 3$: $y[3] = x[2]h[1] = 1 \times 1 = 1$

A sequência resultante é $y[n] = \{1, 3, 3, 1\}$.

9.4. Simulação

Para a implementação computacional, utilizaremos a função `numpy.convolve` para verificar o nosso cálculo manual. Criaremos também uma versão modificada da resposta ao impulso, $h_{mod}[n] = \{1, -1\}$, para repetir o experimento e analisar o impacto na saída.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# sinais originais
x = np.array([1, 2, 1])
h = np.array([1, 1])

# sinal modificado
h_mod = np.array([1, -1])

# cálculo da convolução
y = np.convolve(x, h)
y_mod = np.convolve(x, h_mod)

# eixos de tempo
n_x = np.arange(len(x))
```

```

n_h = np.arange(len(h))
n_y = np.arange(len(y))

# verificação numérica
print(f"Cálculo Computacional y[n]: {y}")
print(f"Cálculo Modificado y_mod[n]: {y_mod}")

# Plotagem dos resultados
fig, axes = plt.subplots(3, 2, figsize=(12, 10))
fig.suptitle('Convolução Discreta: Experimento
Original e Modificado', fontsize=14)

# original
axes[0, 0].stem(n_x, x, linefmt='b-', markerfmt='bo',
basefmt='k-')
axes[0, 0].set_title(r'Entrada:  $x[n] = \{1, 2, 1\}$ ')
axes[0, 0].grid(True)

axes[1, 0].stem(n_h, h, linefmt='g-', markerfmt='go',
basefmt='k-')
axes[1, 0].set_title(r'Resposta ao Impulso Original:
 $h[n] = \{1, 1\}$ ')
axes[1, 0].grid(True)

axes[2, 0].stem(n_y, y, linefmt='r-', markerfmt='ro',
basefmt='k-')
axes[2, 0].set_title(r'Saída Original:  $y[n] = x[n] * h[n]$ ')
axes[2, 0].set_xlabel('n (amostras)')
axes[2, 0].grid(True)

# modificado
axes[0, 1].stem(n_x, x, linefmt='b-', markerfmt='bo',
basefmt='k-')
axes[0, 1].set_title(r'Entrada:  $x[n] = \{1, 2, 1\}$ ')
axes[0, 1].grid(True)

axes[1, 1].stem(n_h, h_mod, linefmt='g-',
markerfmt='gD', basefmt='k-')
axes[1, 1].set_title(r'Resposta ao Impulso Modificada:
 $h_{mod}[n] = \{1, -1\}$ ')
axes[1, 1].grid(True)

axes[2, 1].stem(n_y, y_mod, linefmt='m-',
markerfmt='mD', basefmt='k-')
axes[2, 1].set_title(r'Saída Modificada:
 $y_{mod}[n] = x[n] * h_{mod}[n]$ ')
axes[2, 1].set_xlabel('n (amostras)')
axes[2, 1].grid(True)

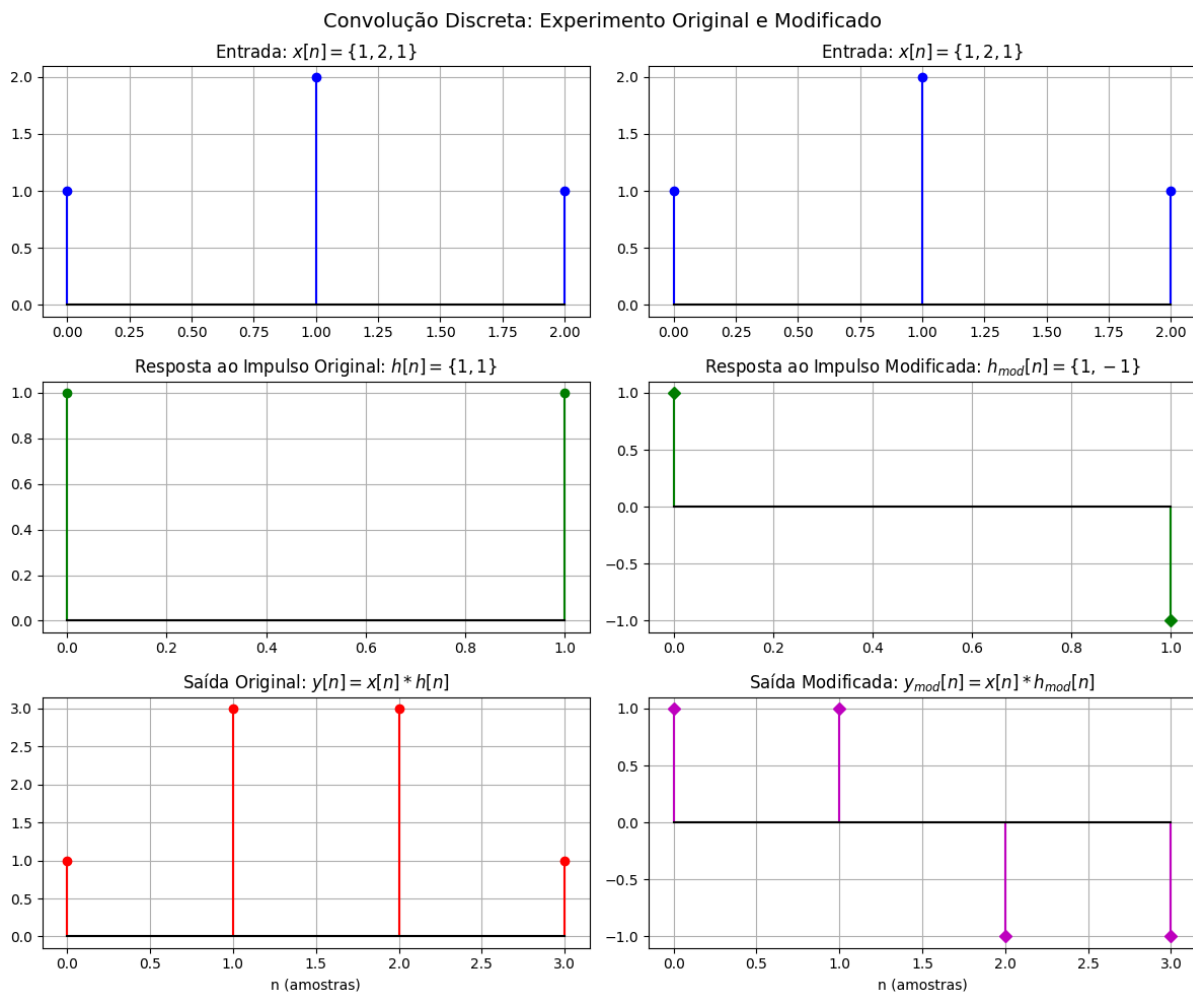
```

```
plt.tight_layout()
plt.show()
```

9.5. Resultados

Cálculo Computacional $y[n]$: [1 3 3 1]

Cálculo Modificado $y_{mod}[n]$: [1 1 -1 -1]



Ao executar o script, o terminal imprime a sequência $y[n] = [1, 3, 3, 1]$, que bate perfeitamente com os quatro valores que calculamos à mão. A segunda impressão devolve $y_{mod}[n] = [1, 1, -1, -1]$. Analisando os gráficos da primeira coluna, observamos que o formato da saída original gerou um pulso mais largo e suave, crescendo gradualmente até 3 e descendo de volta para 1. Na segunda coluna, com a resposta ao impulso modificada, o gráfico da saída transita abruptamente de valores positivos para negativos, perdendo o formato arredondado original e exibindo uma inversão de polaridade a meio do sinal.

9.6. Discussões

Matematicamente, a operação de convolução decorre de uma sequência mecânica de quatro passos: a inversão temporal da resposta ao impulso ($h[-k]$), o seu deslocamento passo a passo ($n - k$), a multiplicação ponto a ponto com as amostras da entrada e, por fim, a soma de todos os produtos resultantes para gerar uma única amostra de saída. A correspondência entre o nosso cálculo manual e o vetor gerado pela simulação comprova a confiabilidade da implementação computacional para analisar o sistema. O impacto de modificar a resposta ao impulso $h[n]$ fica bastante evidente ao comparar as duas colunas do experimento. Ao introduzirmos um valor negativo em $h_{mod}[n] = \{1, -1\}$, o sistema LTI deixou de atuar como um acumulador de soma progressiva e passou a atuar como um sensor de diferenças, subtraindo os valores consecutivos da entrada e causando variações bruscas na saída. Isto demonstra que alterações diretas nos coeficientes de $h[n]$ redefinem completamente a forma como o sinal é filtrado e processado pelo sistema.

10. Convolução como Filtragem

10.1. Resumo conceitual

Uma das aplicações mais essenciais da convolução na engenharia é a filtragem de sinais. Em vez de utilizarmos a convolução apenas para prever o comportamento de um sistema, podemos projetar intencionalmente a resposta ao impulso ($h[n]$) do sistema para remover componentes indesejadas de um sinal (como ruído térmico ou interferência elétrica) e preservar a informação útil. O filtro de média móvel é um dos filtros digitais FIR (Finite Impulse Response) mais simples e utilizados para suavização, operando como um filtro passa-baixas básico.

10.2. Formulação matemática

Um filtro de média móvel com M coeficientes possui uma resposta ao impulso constante e distribuída de forma uniforme:

$$h[n] = \frac{1}{M} \{1, 1, \dots, 1\}$$

Em termos práticos, para uma implementação causal com $M = 3$, a saída $y[n]$ é simplesmente a média aritmética da amostra atual e das duas amostras anteriores:

$$y[n] = \frac{x[n] + x[n-1] + x[n-2]}{3}$$

A operação generalizada para a saída filtrada é calculada pela convolução padrão:

$$y[n] = x[n] * h[n]$$

10.3. Exemplo

Num ambiente hospitalar, o sinal de um eletrocardiograma (ECG) capta os batimentos cardíacos do paciente (o sinal original desejado), mas os cabos também funcionam

como antenas, captando ruído elétrico da rede de 60Hz e contrações musculares espúrias (o ruído). Ao passar este sinal contaminado por um filtro de média móvel num microcontrolador, os picos rápidos e aleatórios do ruído são achatados pela média, revelando a onda contínua e suave do coração.

10.4. Simulação

Para investigar o compromisso entre suavização e distorção temporal, vamos gerar um sinal senoidal limpo, adicionar ruído aleatório e processá-lo com filtros de média móvel de $M = 3$, $M = 5$ e $M = 10$. Utilizaremos o modo `same` na convolução do Numpy apenas para manter o eixo temporal alinhado visualmente.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# geração do sinal original e do ruído
n = np.arange(0, 100)
sinal_limpo = np.sin(2 * np.pi * 0.02 * n)
ruído = 0.3 * np.random.randn(len(n))
sinal_contaminado = sinal_limpo + ruído

# valores de M
M_valores = [3, 5, 10]

fig, axes = plt.subplots(4, 1, figsize=(10, 12),
sharex=True)
fig.suptitle('Filtragem por Média Móvel: Compromisso
de Suavização', fontsize=14)

axes[0].plot(n, sinal_limpo, 'g-', linewidth=2,
label='Sinal Limpo Original')
axes[0].plot(n, sinal_contaminado, 'r-', alpha=0.6,
label='Sinal Contaminado (com ruído)')
axes[0].set_title(r'Sinal de Entrada:  $x[n]$ ')
axes[0].legend()
axes[0].grid(True)

for i, M in enumerate(M_valores):
    h_M = np.ones(M) / M

    # filtragem através da convolução
    sinal_filtrado = np.convolve(sinal_contaminado,
h_M, mode='same')

    ax = axes[i+1]
    ax.plot(n, sinal_limpo, 'g--', alpha=0.5,
label='Referência Limpa')
```

```

ax.plot(n, sinal_filtrado, 'b-', linewidth=2,
label=f'Sinal Filtrado')

titulo = r'Filtro Média Móvel $M=' + str(M) + r'$
| $h[n] = \frac{1}{' + str(M) + r'} \{1, ..., 1\}$'
ax.set_title(titulo)

ax.legend(loc='upper right')
ax.grid(True)

axes[-1].set_xlabel('n (amostras)')
plt.tight_layout()
plt.show()

fig2, axes2 = plt.subplots(1, 3, figsize=(12, 3),
sharey=True)
fig2.suptitle('Respostas ao Impulso dos Filtros:  $h[n]$ ',
fontsize=12)

for i, M in enumerate(M_valores):
    h_M = np.ones(M) / M
    n_h = np.arange(M)
    axes2[i].stem(n_h, h_M, linefmt='k-',
markerfmt='ko', basefmt='k-')
    axes2[i].set_title(r'$M=' + str(M) + r'$')
    axes2[i].grid(True)

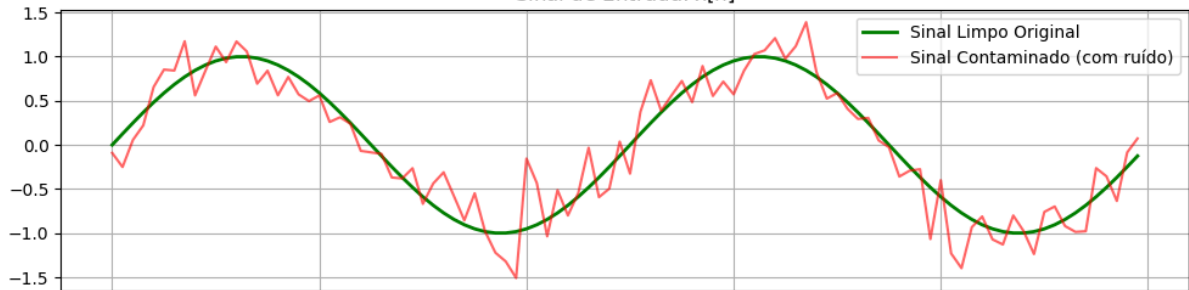
axes2[0].set_ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

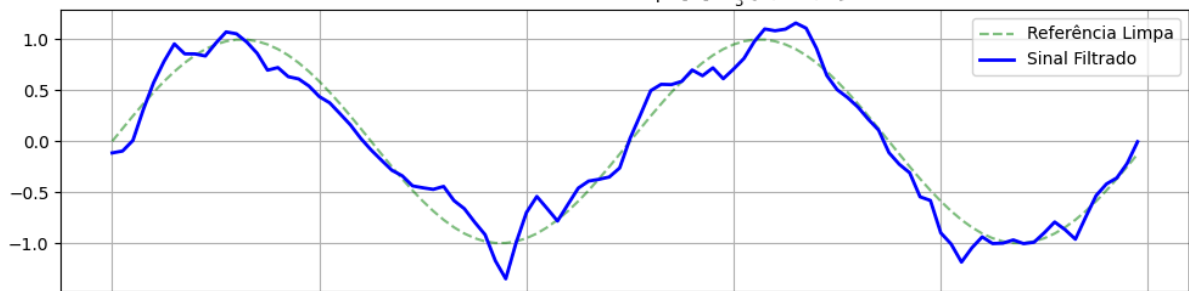
10.5. Resultados

Filtragem por Média Móvel: Compromisso de Suavização

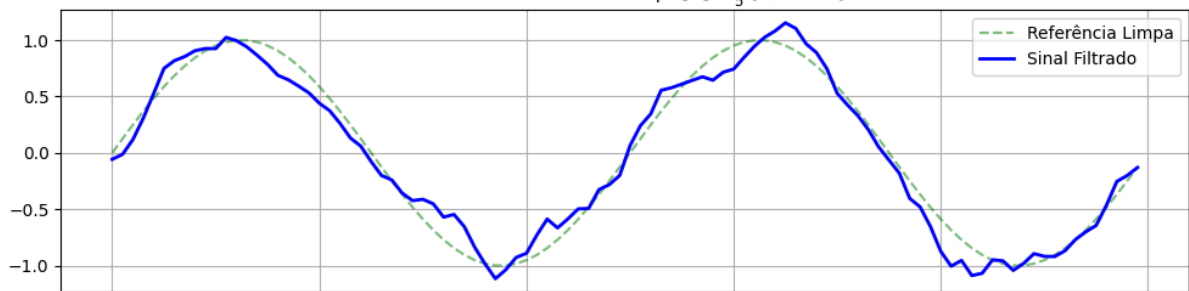
Sinal de Entrada: $x[n]$



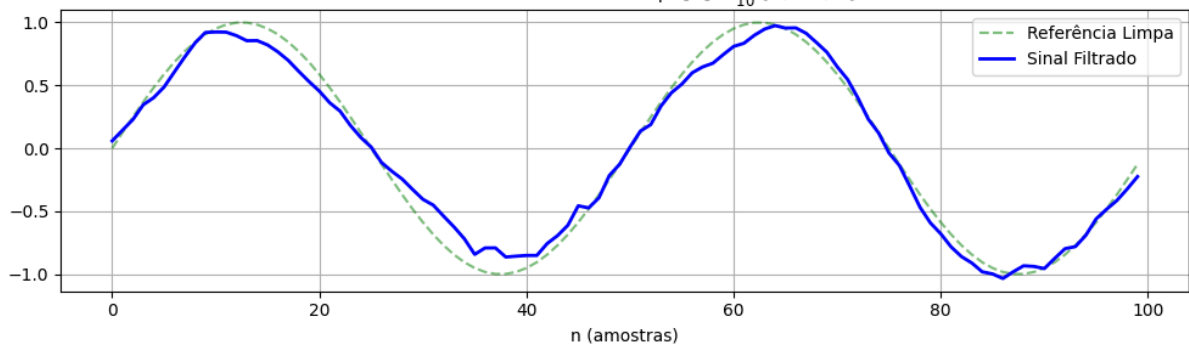
Filtro Média Móvel $M = 3$ | $h[n] = \frac{1}{3}\{1, \dots, 1\}$



Filtro Média Móvel $M = 5$ | $h[n] = \frac{1}{5}\{1, \dots, 1\}$



Filtro Média Móvel $M = 10$ | $h[n] = \frac{1}{10}\{1, \dots, 1\}$





Gráficos da Resposta ao Impulso ($h[n]$): Revelam blocos retangulares de amplitude decrescente. Para $M = 3$, temos 3 amostras com amplitude 0,33. Para $M = 10$, temos 10 amostras com amplitude 0,10.

Gráficos dos Sinais: O primeiro gráfico ilustra a onda senoidal corrompida por sobressaltos (ruído).

- Com $M = 3$: O sinal apresenta uma leve suavização, mas ainda carrega muita vibração aleatória acompanhando a onda de referência.
- Com $M = 5$: O ruído é consideravelmente atenuado, resultando num traçado visivelmente mais limpo.
- Com $M = 10$: A onda perde quase totalmente a textura do ruído, mas começa a sofrer um achatamento da sua amplitude máxima (os picos não atingem mais o topo da referência verde) e observa-se um notório atraso de fase (a onda azul parece empurrada para a direita face à onda verde).

10.6. Discussões

A investigação evidencia o principal dilema (*trade-off*) relacionado ao uso de filtros no processamento de sinais. Nesse sistema, a operação de convolução distribui o efeito de uma irregularidade súbita entre várias amostras vizinhas. Com o aumento do valor de M , o filtro considera um intervalo maior de amostras anteriores para calcular a média, o que melhora sua capacidade de reduzir ruídos aleatórios.

Por outro lado, esse aumento também pode comprometer as características temporais mais rápidas do sinal de interesse. O filtro passa a responder de forma mais lenta às mudanças reais da grandeza física, gerando um atraso no processamento e reduzindo a amplitude dos picos presentes no sinal original. Dessa forma, o projeto do filtro exige um equilíbrio na escolha de M , de modo que seja possível reduzir o ruído sem comprometer as características importantes do sinal.

11. Mini Projeto Integrador

11.1. Escolha do fenômeno

Para o desenvolvimento deste projeto integrador, a equipe optou por simular a aquisição e o processamento de um sinal de áudio. Mais especificamente, será modelado um acorde musical simples, composto por duas notas distintas tocadas simultaneamente. A escolha desse fenômeno se deve à sua composição espectral e às suas variações ao longo do tempo. Além disso, os sinais de áudio permitem observar de forma clara, tanto visualmente quanto matematicamente, os efeitos da amostragem, da perda de resolução causada pela quantização e da aplicação de um filtro digital para a redução de ruídos.

11.2. Modelagem do sinal

Fisicamente, o som propaga-se no ar através de ondas mecânicas de variação de pressão, executando um movimento harmônico que é descrito na matemática por funções trigonométricas. Portanto, o fenômeno acústico escolhido será modelado através da combinação linear de dois sinais senoidais contínuos no tempo, simulando a síntese de duas notas tocadas em simultâneo. A equação que rege este comportamento segue o formato combinado sugerido no roteiro e é definida por:

$$x(t) = A_1 \cos(2\pi f_1 t) + A_2 \cos(2\pi f_2 t)$$

O significado físico dos parâmetros utilizados no modelo é o seguinte:

- Freqüências (f_1 e f_2): Representam a altura (tom) das notas musicais geradas. Adotamos $f_1 = 440\text{Hz}$, que corresponde à freqüência fundamental da nota Lá (A4), e $f_2 = 554\text{Hz}$, correspondente à nota Dó sustenido (C#5). A combinação de ambas forma um intervalo de terça maior.
- Amplitudes (A_1 e A_2): Representam a intensidade acústica (volume) de cada nota que compõe o acorde. Definimos $A_1 = 1.0$ e $A_2 = 0.6$, indicando que a nota fundamental (Lá) possui maior intensidade na composição do sinal.
- Tempo (t): Representa a variável contínua independente que dita a evolução temporal do som, medida em segundos. A fase de ambas as ondas foi considerada nula para simplificar a origem temporal do sinal.

11.3. Geração computacional

O sinal contínuo foi gerado computacionalmente em linguagem Python, estabelecendo uma densidade temporal extremamente elevada para emular com precisão a continuidade física da onda sonora. Os parâmetros definidos para a geração foram:

- Modelo matemático: $x(t) = A_1 \cos(2\pi f_1 t) + A_2 \cos(2\pi f_2 t)$
- Amplitudes: $A_1 = 1.0$ e $A_2 = 0.6$
- Freqüências: $f_1 = 440\text{Hz}$ e $f_2 = 554\text{Hz}$
- Fase: $\phi = 0$ para ambas as componentes.
- Intervalo de observação: Escolheu-se um intervalo muito curto, de 0 a 0,01 segundos (10 *milissegundos*), garantindo que seja possível observar visualmente as oscilações completas do sinal sem sobrecarregar o gráfico com milhares de ciclos idênticos.

Código

```
import numpy as np
import matplotlib.pyplot as plt

# parâmetros do sinal de áudio
f1 = 440.0 # Nota Lá - A4
f2 = 554.0 # Nota Dó# - C#5
A1 = 1.0
```

```

A2 = 0.6
fase = 0

# intervalo de observação (10 milissegundos)
t_limite = 0.01

t_continuo = np.linspace(0, t_limite, 2000)
x_continuo = A1 * np.cos(2 * np.pi * f1 * t_continuo + fase) + \
             A2 * np.cos(2 * np.pi * f2 * t_continuo + fase)

```

11.4. Amostragem

Para digitalizar o sinal contínuo, é necessário definir uma frequência de amostragem (f_s). Como a maior frequência presente no nosso sinal é de 554 Hz , a teoria do processamento de sinais exige que a amostragem seja pelo menos o dobro desse valor para evitar perdas ou distorção. Para garantir uma boa resolução digital, optou-se por $f_s = 8000\text{ Hz}$ (uma taxa clássica utilizada em sistemas de telefonia digital). A partir desta frequência, determina-se o período de amostragem:

$$T_s = \frac{1}{f_s} = \frac{1}{8000} = 0,000125 \text{ segundos}$$

O sinal discreto resultante é representado pela extração das amostras em intervalos regulares:

$$x[n] = x(nT_s)$$

11.4.1. Código

```

# amostragem
fs = 8000.0
Ts = 1 / fs

# eixo temporal discreto
t_discreto = np.arange(0, t_limite, Ts)
n_amostras = np.arange(len(t_discreto))

# sinal discretizado
x_discreto = A1 * np.cos(2 * np.pi * f1 * t_discreto + fase) + \
            A2 * np.cos(2 * np.pi * f2 * t_discreto + fase)

fig, axes = plt.subplots(2, 1, figsize=(10, 8))

axes[0].plot(t_continuo, x_continuo, 'b-', label='Sinal Contínuo: x(t)')
axes[0].plot(t_discreto, x_discreto, 'ro', label='Instantes de Amostragem')

```

```

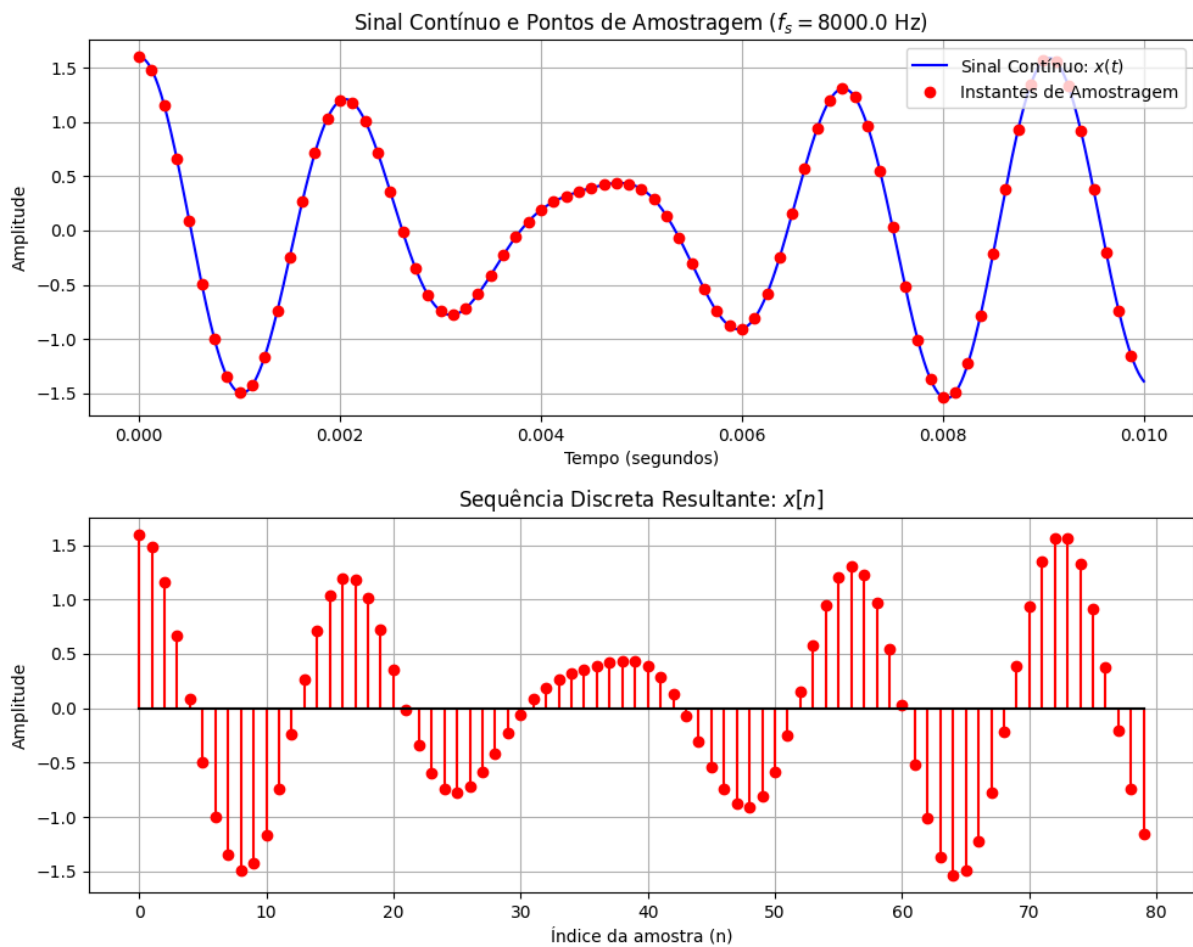
axes[0].set_title(rf'Sinal Contínuo e Pontos de Amostragem ( $f_s = f_s$ 
Hz) ')
axes[0].set_xlabel('Tempo (segundos)')
axes[0].set_ylabel('Amplitude')
axes[0].grid(True)
axes[0].legend(loc='upper right')

axes[1].stem(n_amostras, x_discreto, linefmt='r-', markerfmt='ro',
basefmt='k-')
axes[1].set_title(r'Sequência Discreta Resultante:  $x[n]$ ')
axes[1].set_xlabel('Índice da amostra (n)')
axes[1].set_ylabel('Amplitude')
axes[1].grid(True)

plt.tight_layout()
plt.show()

```

11.4.2. Resultados



11.5. Quantização

Após a discretização temporal, o sinal de áudio precisa ter a sua amplitude convertida para um conjunto finito de valores, processo conhecido como quantização. A amplitude máxima teórica do nosso sinal é a soma das amplitudes ($1.0 + 0.6 = 1.6$). Assumindo um sistema que opera numa faixa de $-2V$ a $+2V$ (excursão total de $4V$), simulou-se a quantização utilizando duas resoluções distintas: $N_1 = 4\text{bits}$ e $N_2 = 8\text{bits}$. Com 4bits , o sistema possui apenas 16 níveis discretos, resultando num passo de quantização grande ($\Delta V = 0,25V$). O sinal quantizado fica com um aspecto de "escadaria" pronunciado, o que no áudio se traduziria num ruído de quantização severo e distorção harmônica. Com 8bits (256 níveis e $\Delta V \approx 0,015V$), a onda quantizada sobre põe-se quase perfeitamente à onda discreta original, preservando a fidelidade acústica do acorde.

11.5.1. Código

```
# faixa de operação do ADC
excursao = 4.0

# quantização com N = 4 bits
L_4 = 2**4
delta_4 = excursao / L_4
x_quant_4 = np.round(x_discreto / delta_4) * delta_4

# quantização com N = 8 bits
L_8 = 2**8
delta_8 = excursao / L_8
x_quant_8 = np.round(x_discreto / delta_8) * delta_8

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
fig.suptitle('Etapa 4: Efeito da Quantização no Sinal de Áudio',
             fontsize=14)

axes[0].plot(n_amstras, x_discreto, 'g--', alpha=0.5, label='Sinal Original x[n]')
axes[0].step(n_amstras, x_quant_4, 'b-', where='mid', label='Quantizado (4 bits)')
axes[0].set_title(r'Quantização com N = 4 bits (16 níveis)')
axes[0].set_ylabel('Amplitude')
axes[0].grid(True)
axes[0].legend()

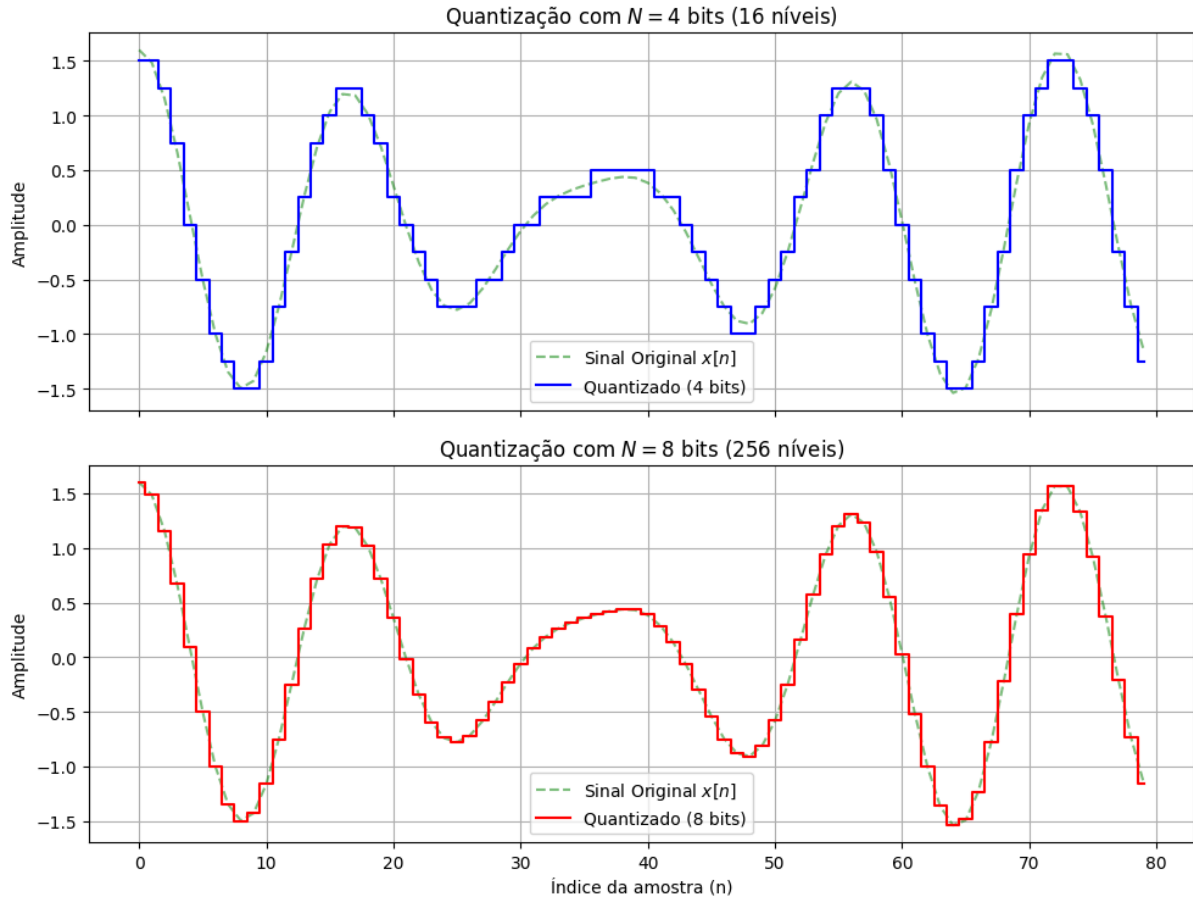
axes[1].plot(n_amstras, x_discreto, 'g--', alpha=0.5, label='Sinal Original x[n]')
axes[1].step(n_amstras, x_quant_8, 'r-', where='mid', label='Quantizado (8 bits)')
axes[1].set_title(r'Quantização com N = 8 bits (256 níveis)')
axes[1].set_xlabel('Índice da amostra (n)')
axes[1].set_ylabel('Amplitude')
axes[1].grid(True)
```

```
axes[1].legend()

plt.tight_layout()
plt.show()
```

11.5.2. Resultados

Etapa 4: Efeito da Quantização no Sinal de Áudio



11.6. Inclusão de ruído

Em sistemas reais de gravação de áudio, o sinal está sempre sujeito a interferências. Para simular esta perturbação, adicionou-se um ruído branco gaussiano ($r[n]$) ao sinal amostrado, originando o sinal contaminado $x_r[n] = x[n] + r[n]$. Justifica-se a escolha do ruído gaussiano por ser o modelo matemático ideal para representar o "chiado" de fundo (ruído térmico) introduzido pelos circuitos eletrônicos do pré-amplificador do microfone. A amplitude do ruído foi definida com um desvio padrão de 0,15 (cerca de 10% da amplitude do sinal principal) para simular uma gravação de baixa qualidade, onde o ruído é visível e audível, mas não oblitera completamente a forma de onda do acorde musical.

11.6.1. Código

```
# geração do ruído
np.random.seed(42)
ruído = 0.15 * np.random.randn(len(x_discreto))

# sinal contaminado
xr_discreto = x_discreto + ruído

fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)
fig.suptitle('Etapa 5: Inclusão de Ruído Branco Gaussiano',
             fontsize=14)

axes[0].plot(n_amostras, x_discreto, 'g-')
axes[0].set_title('1. Sinal Original Discreto:  $x[n]$ ')
axes[0].grid(True)

axes[1].plot(n_amostras, ruído, 'k-', alpha=0.7)
axes[1].set_title('2. Ruído Branco:  $r[n]$ ')
axes[1].set_ylabel('Amplitude')
axes[1].grid(True)

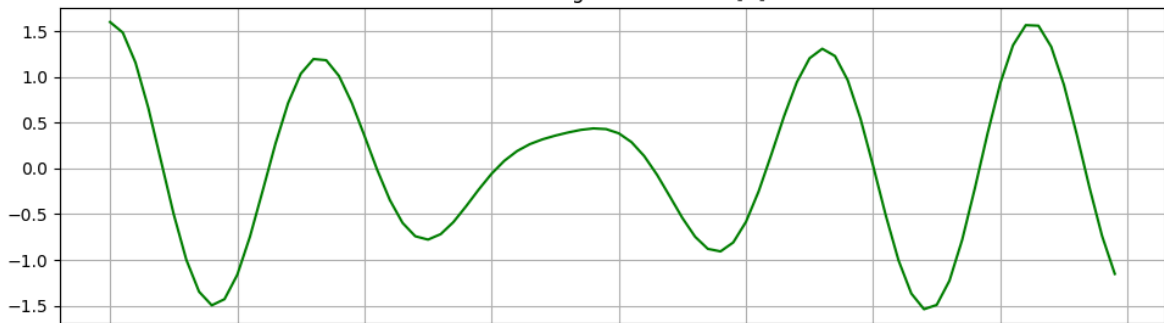
axes[2].plot(n_amostras, xr_discreto, 'r-')
axes[2].set_title('3. Sinal Contaminado:  $x_r[n] = x[n] + r[n]$ ')
axes[2].set_xlabel('Índice da amostra (n)')
axes[2].grid(True)

plt.tight_layout()
plt.show()
```

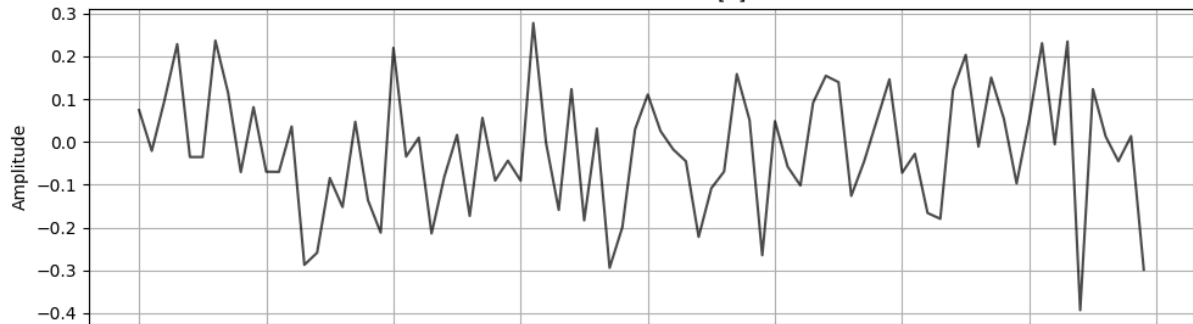
11.6.2. Resultados

Etapa 5: Inclusão de Ruído Branco Gaussiano

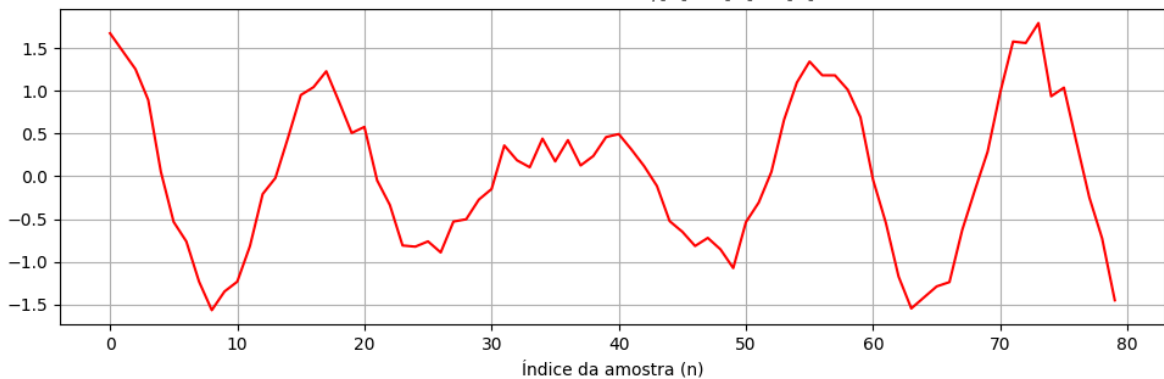
1. Sinal Original Discreto: $x[n]$



2. Ruído Branco: $r[n]$



3. Sinal Contaminado: $x_r[n] = x[n] + r[n]$



11.7. Processamento por Sistema LTI

Para atenuar a degradação causada pelo ruído gaussiano, o sinal contaminado foi processado através de um sistema LTI atuando como filtro digital. A escolha recaiu sobre o filtro de média móvel com $M = 5$ coeficientes. A justificativa para este valor de M baseia-se no compromisso entre suavização e fidelidade do áudio: $M = 5$ equivale a uma janela temporal de $5 \times 0,000125 = 0,625$ *milissegundos*. Esta janela é larga o suficiente para esmagar os picos de chiado de alta frequência, mas curta o suficiente para não distorcer

severamente a onda de 554 Hz do acorde (cujo ciclo completo dura cerca de 1,8 milissegundos). A resposta ao impulso do sistema é $h[n] = \frac{1}{5}\{1, 1, 1, 1, 1\}$.

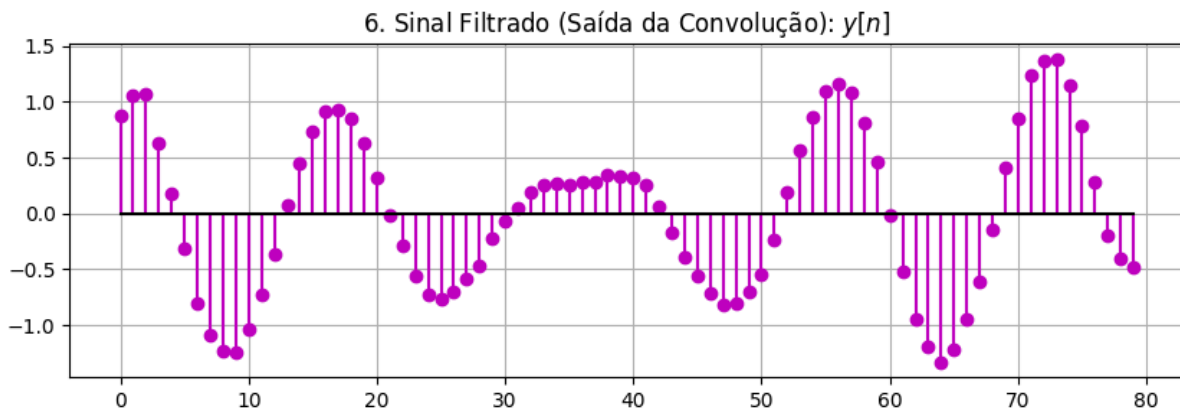
11.7.1. Código

```
# sistema LTI
M = 5
h_n = np.ones(M) / M

# convolução
y_n = np.convolve(xr_discreto, h_n, mode='same')

plt.figure(figsize=(10, 3))
plt.stem(n_amstras, y_n, linefmt='m-', markerfmt='mo', basefmt='k-')
plt.title(r'6. Sinal Filtrado (Saída da Convolução): y[n]')
plt.grid(True)
plt.show()
```

11.7.2. Resultados



11.8. Comparação de Resultados

A simulação computacional gerou um painel gráfico consolidado, permitindo a comparação visual de todas as fases da cadeia de aquisição e processamento do sinal de áudio, desde o modelo matemático contínuo original até ao sinal discretizado, quantizado, corrompido por ruído e, por fim, recuperado pela convolução com o sistema LTI.

11.8.1. Código

```
# comparação dos resultados
fig, axes = plt.subplots(7, 1, figsize=(10, 20), sharex=False)
fig.suptitle('Etapa 7: Cadeia Completa de Aquisição e Processamento do Sinal', fontsize=16, y=0.92)
```

```

# sinal original
axes[0].plot(t_continuo, x_continuo, 'b-')
axes[0].set_title('1. Sinal Contínuo Original:  $x(t)$ ')
axes[0].grid(True)

# sinal amostrado
axes[1].stem(n_amostras, x_discreto, linefmt='g-', markerfmt='go',
basefmt='k-')
axes[1].set_title('2. Sinal Amostrado:  $x[n]$ ')
axes[1].grid(True)

# sinal quantizado
axes[2].step(n_amostras, x_quant_8, 'c-', where='mid')
axes[2].set_title('3. Sinal Quantizado (8 bits)')
axes[2].grid(True)

# sinal com ruído
axes[3].plot(n_amostras, xr_discreto, 'r-')
axes[3].set_title('4. Sinal Contaminado com Ruído Gaussiano:  $x_r[n]$ ')
axes[3].grid(True)

# resposta ao impulso
n_h = np.arange(len(h_n))
axes[4].stem(n_h, h_n, linefmt='k-', markerfmt='ko', basefmt='k-')
axes[4].set_title(f'5. Resposta ao Impulso (Média Móvel  $M=\{M\}$ ):  $h[n]$ ')
axes[4].set_xlim(-1, 10)
axes[4].grid(True)

# sinal após a convolução
axes[5].plot(n_amostras, y_n, 'm-', linewidth=2)
axes[5].set_title('6. Sinal Filtrado (Saída da Convolução):  $y[n]$ ')
axes[5].grid(True)

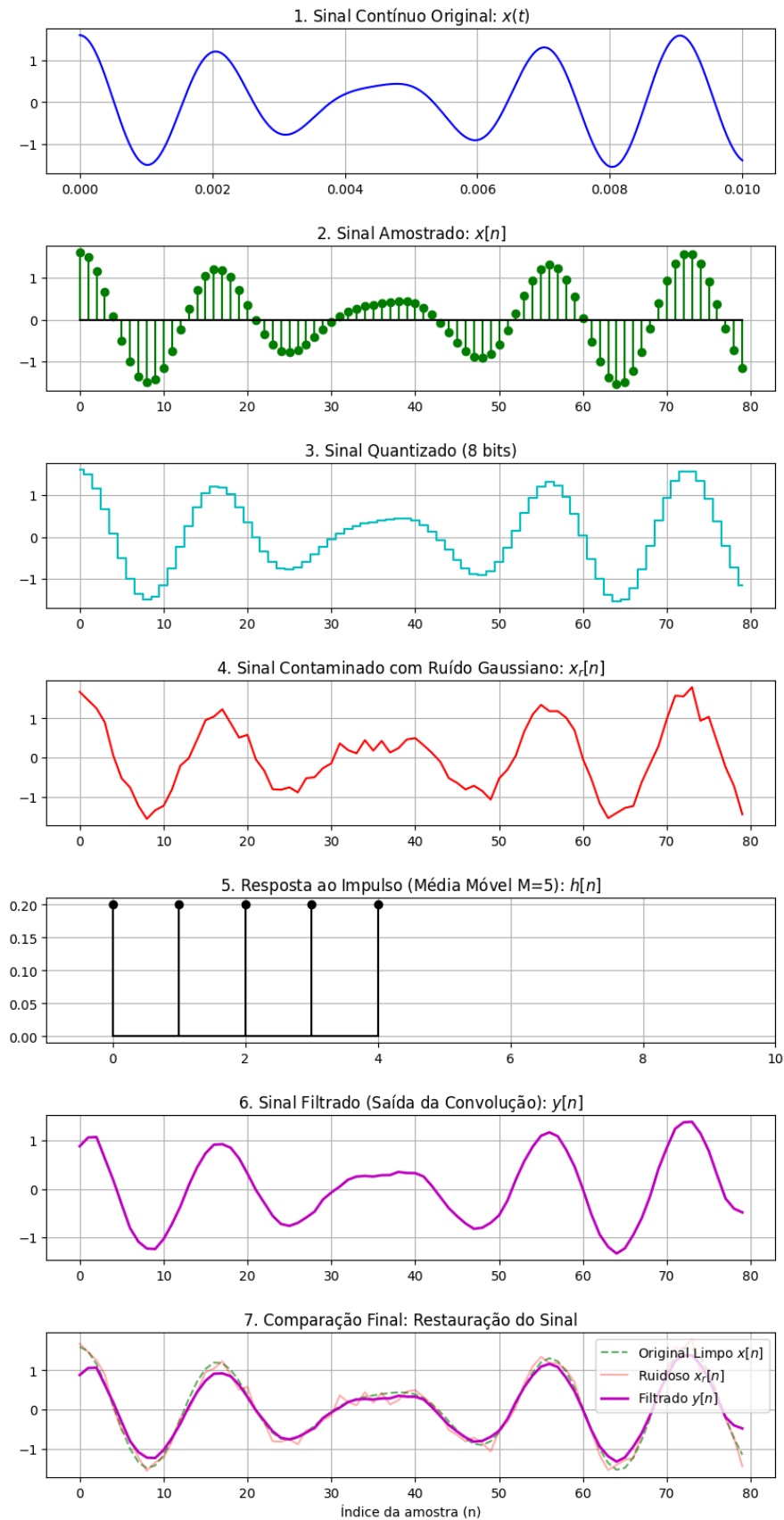
# comparação entrada vs saída
axes[6].plot(n_amostras, x_discreto, 'g--', alpha=0.6,
label='Original Limpo  $x[n]$ ')
axes[6].plot(n_amostras, xr_discreto, 'r-', alpha=0.3, label='Ruidoso  $x_r[n]$ ')
axes[6].plot(n_amostras, y_n, 'm-', linewidth=2, label='Filtrado  $y[n]$ ')
axes[6].set_title('7. Comparação Final: Restauração do Sinal')
axes[6].set_xlabel('Índice da amostra (n)')
axes[6].legend(loc='upper right')
axes[6].grid(True)

plt.subplots_adjust(hspace=0.5)
plt.show()

```

11.8.2. Resultados

Etapa 7: Cadeia Completa de Aquisição e Processamento do Sinal



11.9. Discussão

O fenômeno acústico foi inicialmente modelado a partir da soma de duas senoides, com frequências de 440 Hz e 554 Hz, representando duas notas musicais tocadas simultaneamente. Para a digitalização do sinal, foi utilizada uma frequência de amostragem de $f_s = 8000$ Hz. Esse valor é superior ao dobro da maior frequência presente no sinal, de acordo com o Teorema de Nyquist, permitindo representar adequadamente as oscilações das duas componentes e evitando a ocorrência de aliasing.

Na etapa de quantização, o eixo das amplitudes foi dividido em níveis discretos, fazendo com que o sinal assumisse valores em degraus. Esse processo introduz um erro de quantização, que se torna mais perceptível quando são utilizados poucos bits. Na simulação, a quantização com apenas 4 bits produziu uma distorção significativa na forma de onda. Com o aumento da resolução para 8 bits, os níveis de amplitude se tornaram mais próximos entre si, reduzindo a distorção e fazendo com que o sinal quantizado se aproximasse mais da forma de onda original.

Para representar a presença de ruídos no sinal, foi adicionada uma componente de ruído gaussiano. Essa perturbação introduziu variações aleatórias ao longo do tempo, simulando um ruído de fundo que poderia ser percebido durante a reprodução do áudio. Como etapa de recuperação do sinal, foi utilizado um filtro com resposta ao impulso dada por $h[n] = 0, 2; 0, 2; 0, 2; 0, 2; 0, 2$. Esse filtro realiza uma média móvel de cinco amostras, considerando a amostra atual e as quatro anteriores. Por utilizar apenas operações de soma e multiplicação por uma constante e manter seus coeficientes fixos, o sistema pode ser caracterizado como Linear e Invariante no Tempo (LTI).

O sinal com ruído foi então processado por meio da convolução com $h[n]$. Como o filtro realiza uma média entre amostras consecutivas, as variações rápidas tendem a ser reduzidas, apresentando um comportamento semelhante ao de um filtro passa-baixas. Dessa forma, foi possível observar uma redução significativa das oscilações provocadas pelo ruído.

Entretanto, a redução do ruído também provocou alterações no sinal de interesse. Na saída do filtro, foi possível observar uma redução na amplitude dos picos e um pequeno deslocamento temporal em relação ao sinal original. Esse comportamento está relacionado às características do filtro de média móvel, que atribui o mesmo peso às amostras utilizadas no cálculo. Assim, embora o filtro reduza as variações de alta frequência associadas ao ruído, ele também pode atenuar componentes de variação rápida que fazem parte do próprio sinal de áudio.

Como uma possível melhoria para o projeto, pode-se utilizar um filtro FIR ou IIR com uma faixa de passagem definida de acordo com as frequências de interesse do sinal. Neste caso, um filtro passa-banda poderia ser projetado para preservar principalmente as componentes entre 200 Hz e 1000 Hz, enquanto atenuaria as frequências abaixo e acima dessa faixa. Dessa forma, seria possível buscar uma redução mais seletiva do ruído, reduzindo a interferência sobre as características do sinal musical.

12. Conclusões

O presente estudo dirigido contribuiu para a compreensão dos principais conceitos relacionados ao Processamento Digital de Sinais, tanto do ponto de vista teórico quanto prático. Ao longo das atividades, foi possível analisar a passagem do sinal contínuo para o domínio discreto, observando os efeitos da amostragem e da quantização nesse processo. O estudo de sistemas Lineares e Invariantes no Tempo (LTI) e da operação de convolução também permitiu compreender melhor o funcionamento e o comportamento dos filtros digitais.

No Mini Projeto Integrador, esses conceitos foram aplicados à aquisição e ao processamento de um sinal de áudio com a presença de ruído. A modelagem de um acorde musical e a aplicação de um filtro LTI permitiram observar, na prática, a relação entre a redução do ruído e a preservação das características do sinal original. Os resultados mostraram que o aumento da capacidade de atenuação do ruído pode também afetar componentes importantes do sinal, como a amplitude dos picos e as variações mais rápidas, além de provocar um pequeno atraso no sinal de saída.

Por fim, a utilização da linguagem Python, juntamente com as bibliotecas NumPy e Matplotlib, foi importante para a realização dos cálculos e para a visualização dos resultados. O uso dessas ferramentas facilitou a análise do comportamento do sinal em cada etapa do processamento e permitiu comparar os resultados obtidos com os conceitos estudados ao longo do trabalho. Dessa forma, a atividade possibilitou relacionar os fundamentos teóricos de Processamento Digital de Sinais com sua aplicação em uma situação prática.

13. Referências

LATHI, B. P. **Sinais e Sistemas Lineares**. 2. ed. Porto Alegre: Bookman, 2007.

OPPENHEIM, A. V.; SCHAFER, R. W. **Processamento Discreto de Sinais**. 3. ed. São Paulo: Pearson Prentice Hall, 2012.

PROAKIS, J. G.; MANOLAKIS, D. K. **Processamento Digital de Sinais: Princípios, Algoritmos e Aplicações**. 4. ed. São Paulo: Pearson, 2007.

HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020. Disponível em: <https://www.nature.com/articles/s41586-020-2649-2>.

HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90-95, 2007. Disponível em: <https://ieeexplore.ieee.org/document/4160265>.